

Technische Universität Berlin

Institut für Softwaretechnik und Theoretische Informatik
Machine Learning Group
Fakultät IV



Master's Thesis

Exploring GNN-LRP for Graph Neural Networks in Android Malware Detection

Ferran Guasch

Matriculation Number: 510349
Berlin, May 2025

Supervised by:

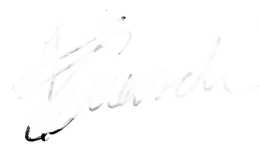
PhD. Thomas Schnake - Technische Universität Berlin
Prof. Dr. Daniel Arp - Technische Universität Wien

Examiners:

Prof. Dr. Klaus-Robert Müller - Technische Universität Berlin
Prof. Dr. Daniel Arp - Technische Universität Wien

Hereby I declare that I wrote this thesis myself with the help of no more than the mentioned literature and auxiliary means.

Berlin, 20.05.2025

A handwritten signature in dark ink, appearing to read 'Guasch', is written above a horizontal dotted line.

(Ferran Guasch)

Acknowledgements

I would like to express my sincere gratitude to my advisors, Thomas Schanke and Daniel Arp, for their invaluable guidance, unwavering support, and insightful feedback throughout this research. Their expertise and encouragement have made this journey both rewarding and memorable.

Abstract

Android malware is becoming increasingly complex and evasive, making traditional detection methods less effective. Graph Neural Networks (GNNs) have recently shown strong performance in malware classification by using structured representations such as function call graphs. However, beyond achieving high classification accuracy, it is equally important to understand the reasoning behind a model’s predictions, both for addressing security vulnerabilities and for building trust in the detection process.

This thesis explores explainable artificial intelligence (XAI) techniques to bridge this gap, focusing on improving interpretability in graph-based malware detection. Specifically, we apply Layer-wise Relevance Propagation (LRP) to GNNs, an approach referred to as GNN-LRP, to assign relevance scores to graph components based on their influence on model predictions. We make use of the MsDroid framework to generate structured representations of each APK in the form of subgraphs derived from the program call graph.

Using a combination of real-world datasets and a custom synthetic dataset, we demonstrate that GNN-LRP effectively highlights semantically meaningful paths indicative of malicious behavior. For instance, within the FakeInstaller malware family, our method successfully identifies the core behavioral structures responsible for malicious activity, such as SMS fraud routines and conditional payload triggers. These results illustrate that GNN-LRP can reveal the internal logic driving model decisions and isolate the functional patterns underlying malware behavior, insights that are difficult to obtain through heuristic-based approaches.

By bridging performance and interpretability, this work contributes a robust and transparent step forward in graph-based malware analysis.

Contents

1	Introduction	1
1.1	Motivation and Outline	2
1.1.1	Motivation	2
1.1.2	Thesis Outline	2
2	Background	3
2.1	Android Malware	3
2.1.1	APK Structure and Malware Characteristics	3
2.1.2	Common Malicious Behavior	3
2.1.3	Android Malware Datasets	4
2.1.4	Feature Extraction and Graph Representations	4
2.2	Graph Neural Networks	5
2.2.1	Fundamentals of GNNs	5
2.2.2	GNNs in Malware Detection and Cybersecurity	6
2.2.3	Graph Isomorphism Networks	6
2.3	GNN-LRP: Interpreting Graph Neural Networks via Relevant Walks	7
2.3.1	Layer-wise Relevance Propagation (LRP)	7
2.3.2	Extending LRP to GNNs: Relevant Walks	8
2.3.3	Why GNN-LRP May Be Well-Suited for Malware Detection	8
2.4	MsDroid: Identifying Malicious Snippets in Android Apps	9
2.4.1	Adopted Components in This Thesis	9
2.4.2	Subgraph Generation Methodology	9
2.4.3	GNN Models for Subgraph Classification	10
2.4.4	Evaluation and Results	10
2.4.5	Explanation Mechanism	10
3	Related Work	12
3.1	Android Malware Detection Techniques	12
3.2	Graph-Based Representations for Android Applications	12
3.3	Graph Neural Networks in Malware Detection	13
3.4	Explainable AI in Security and Malware Analysis	14
3.5	GNN Explainability: Techniques and Applications	14
4	Methodology	16
4.1	Datasets	16
4.1.1	Synthetic Dataset	16
4.1.2	Drebin Dataset	18
4.1.3	Androzoo Dataset	18
4.1.4	Dataset Summary	18
4.2	GNN-LRP Implementation	19
4.2.1	Overview	19
4.2.2	Class Design and Implementation	19

4.2.3	Message Passing Relevance Rule	20
4.2.4	Linear Layers	21
4.2.5	Pooling Layers	21
4.2.6	GNN-LRP Visualization	21
4.3	MsDroid Subgraph Creation	22
4.4	Evaluation Framework	24
4.4.1	Correctness	24
4.4.2	Completeness	25
4.4.3	Contrastivity	25
4.4.4	Continuity	25
4.4.5	Consistency	26
4.4.6	Compactness	26
4.4.7	Coherence	26
5	Results	28
5.1	Synthetic Dataset	28
5.1.1	Experimental Setup	28
5.1.2	MsDroid Subgraph Explanation Method	28
5.1.3	GNN-LRP on Test Set	29
5.1.4	Insights from Training Data	30
	Relevance of openConnection	31
	Relevance of prepareMaliciousPayload	31
5.1.5	GNN-LRP Evaluation	32
	Correctness, Completeness, Continuity, and Contrastivity	32
	Consistency, Compactness, and Coherence	35
5.2	Case Study: FakeInstaller Malware Family	38
5.2.1	Background on FakeInstaller	38
5.2.2	Training and Inference Setup	38
5.2.3	Analysis Overview	38
	startC2DM and SIM ID Checks	39
	addSEND7SMS & fillPostData	40
	fillSMSInfo & loadMCCMNCFile	41
	sendSMSKa	42
6	Conclusions and Future Work	45
6.1	Conclusions	45
6.2	Future Work	45
A	Supplementary Tables	47
A.1	Subgraph Statistics for Drebin Dataset	47
B	Additional Experimental Details	51
B.1	Hashes of Synthetic Dataset Base APKs	51

List of Figures

2.1	Example of a function call graph extracted from an Android APK. Nodes represent functions; edges represent method invocations.	5
2.2	MsDroid subgraph generation from [15]	10
4.1	Simplified call graph representing the injected malware behavior.	17
4.2	GNN-LRP visualizations on a sample subgraph. Left: walk-level relevance plot Center: edge-level relevance summary. Right: node-level relevance summary.	22
4.3	Example of a subgraph in the original MsDroid representation	23
4.4	Improved subgraph showing merged nodes and recursive paths	24
5.1	MsDroid explanation – malicious subgraph 2EE114A	29
5.2	MsDroid explanation – benign subgraph falsely identified as malicious 35A4BEFD	29
5.3	GNN-LRP edge-level relevance visualization.	30
5.4	GNN-LRP node-level relevance visualization.	30
5.5	GNN-LRP walk-level relevance visualization.	30
5.6	Benign training sample ED0F0D25	31
5.7	Overfitted benign training sample ED0F0D25	31
5.8	Benign training sample 8D8C671	31
5.9	Benign training sample A4512013	31
5.10	Explanations generated by GNN-LRP with randomized model weights. Relevance is distributed uniformly and lacks semantic focus, as expected.	32
5.11	Explanations generated by GNN-LRP after training on randomized labels. Although relevance is assigned, the highlighted structures lack correspondence with known malicious behavior, confirming dependence on meaningful supervision.	33
5.12	Incremental edge deletion on six subgraphs. Red line: deletion by relevance score. Blue line: random edge deletion. A faster drop in the red line indicates that the explanation method correctly identifies key decision-driving edges.	34
5.13	Explanation subgraphs from three malware samples using a 4-layer GNN. The core behavioral structure is preserved across samples, indicating explanation consistency.	36
5.14	Subgraphs exhibiting the pattern <code>startC2DM</code> + SIM ID checks (<code>isMegafon</code> , <code>isUKID</code> , etc.). These patterns likely trigger malicious logic conditionally based on device characteristics and enable remote payload delivery. (SHA prefixes: <code>e6b75d</code> , <code>5c4121</code> , <code>82e92d</code>)	39
5.15	Detailed example of the <code>startC2DM</code> pattern. (SHA prefix: <code>1a1bb5</code>)	40
5.16	Example subgraphs where <code>addSEND7SMS</code> and <code>fillPostData</code> are central. (SHA prefixes: <code>053323</code> , <code>b52df7</code> , <code>bef54e</code>)	41
5.17	Detailed example of the <code>addSEND7SMS</code> pattern. (SHA prefix: <code>a2438d</code>)	41

5.18	GNN-LRP explanations for subgraphs involving <code>fillSMSInfo</code> and <code>loadMCCMNCFFile</code> , showing how network-code loading and SMS harvesting routines are interconnected. (SHA prefixes: 69bc6b, 728bc8, e9cee4)	42
5.19	Detailed example of the <i>fillSMSInfo</i> pattern. (SHA prefix: 004d92)	42
5.20	Isolated SMS sending structures centered around <code>sendSMSka</code> and its variants. The activation pattern is controlled via <code>checkCommand</code> , often influenced by strings or remote input. (SHA prefixes: 5b54e2, 65a536, 9d3f58)	43
5.21	Detailed example of the <i>sendSMSka</i> pattern. (SHA prefix: 65a536)	43

List of Tables

4.1	Dataset statistics. <i>APK</i> = total APKs, <i>M-APK</i> = malicious APKs, <i>SG</i> = total subgraphs, <i>M-SG</i> = malicious subgraphs, <i>N/S</i> = avg. nodes per subgraph, <i>E/S</i> = avg. edges per subgraph, <i>Deg.</i> = avg. degree, <i>Dens.</i> = avg. density, <i>Size</i> = avg. file size (MB).	19
5.1	Structural explanation metrics across three samples. Low compactness and high entropy confirm that GNN-LRP explanations are sparse, diverse, and stable across model variations.	36
A.1	Dataset statistics (Part 4). <i>APK</i> = total APKs, <i>SG</i> = total subgraphs, <i>N/S</i> = avg. nodes per subgraph, <i>E/S</i> = avg. edges per subgraph, <i>Deg.</i> = avg. degree, <i>Dens.</i> = avg. density, <i>Size</i> = avg. file size (MB).	47
A.2	Dataset statistics (Part 1). <i>APK</i> = total APKs, <i>SG</i> = total subgraphs, <i>N/S</i> = avg. nodes per subgraph, <i>E/S</i> = avg. edges per subgraph, <i>Deg.</i> = avg. degree, <i>Dens.</i> = avg. density, <i>Size</i> = avg. file size (MB).	48
A.3	Dataset statistics (Part 2). <i>APK</i> = total APKs, <i>SG</i> = total subgraphs, <i>N/S</i> = avg. nodes per subgraph, <i>E/S</i> = avg. edges per subgraph, <i>Deg.</i> = avg. degree, <i>Dens.</i> = avg. density, <i>Size</i> = avg. file size (MB).	49
A.4	Dataset statistics (Part 3). <i>APK</i> = total APKs, <i>SG</i> = total subgraphs, <i>N/S</i> = avg. nodes per subgraph, <i>E/S</i> = avg. edges per subgraph, <i>Deg.</i> = avg. degree, <i>Dens.</i> = avg. density, <i>Size</i> = avg. file size (MB).	50

List of Abbreviations

APK	Android Package Kit
GINs	Graph Isomorphism Networks
GNNs	Graph Neural Networks
LRP	Layer-wise Relevance Propagation
ML	Machine Learning
MLP	Multilayer perceptron
XAI	Explainable Artificial Intelligence

1 Introduction

The rapid evolution and increasing complexity of Android malware have made traditional signature-based detection techniques less effective. Attackers continually develop new techniques to bypass both static and dynamic analysis, making the detection of malicious applications increasingly challenging. In response to these challenges, researchers have turned to machine learning approaches, particularly graph neural networks (GNNs), which have shown strong performance in malware detection. These models are especially well-suited to Android applications because they can learn from structured representations like function call graphs — graphical representations of function invocations within the code — capturing both relationships and semantic patterns in the code [37, 9].

Several studies have demonstrated that GNNs can accurately identify whether an application is malicious [15, 19, 11]. However, in practice, detection alone is not always enough. Security practitioners often need to understand why a model made a specific prediction. This helps verify the model’s reasoning, reduce false positives, and potentially locate the source of the malicious behavior, such as the exploitation of a known vulnerability in the Android system. Achieving this level of interpretability is particularly challenging due to the complexity of both the models and the problem domain. Although many explanation methods have been proposed for a wide range of models, truly effective interpretability for high-performing malware detection models remains an open research problem [27, 21].

To address the challenge of interpretability in GNN-based detection systems, researchers have begun exploring explanation methods such as GNN-LRP, introduced by Schnake et al. [29]. GNN-LRP assigns relevance scores to edges within the graph, highlighting the paths the model deems most influential in its prediction. Beyond individual edges, it also enables the computation of relevant walks—sequences of connected nodes and edges—that reflect how information flows through the graph. This makes it possible to visualize and understand higher-order dependencies, showing how the relevance of a particular node propagates along specific chains of interactions. Such insights reveal how choices made at one layer of the GNN can influence downstream nodes, offering a more comprehensive view of the model’s decision-making process. As an explanation method, it’s important to acknowledge that GNN-LRP’s effectiveness is inherently tied to the quality of both the underlying GNN model and the input graph representation. In this work, we apply this explanation technique within the MsDroid framework [15], which provides tools for decompiling Android apps and extracting function call graphs for analysis. These graphs are then used to train a GNN, and GNN-LRP is subsequently employed to interpret the resulting predictions.

1.1 Motivation and Outline

1.1.1 Motivation

This thesis is motivated by the need to improve the interpretability of graph-based malware detection systems. While GNNs have shown high accuracy in classifying Android applications, their lack of transparency limits their practical use in security workflows. Without insight into a model's decision-making process, it is difficult to assess whether it is identifying meaningful patterns of malicious behavior or simply learning to exploit biases in the dataset. Understanding why the model made that incorrect prediction is crucial for debugging the model and preventing future errors.

By applying an explanation method like GNN-LRP to malware detection, we aim to make these models more understandable and trustworthy. This includes evaluating whether the model relies on generalizable behavior patterns, whether the explanations are useful for identifying important parts of the code, and whether similar malicious structures appear across different samples. In doing so, we also hope to uncover patterns that might correspond to new or evolving forms of malware, ultimately contributing to more robust and explainable tools for Android malware analysis.

1.1.2 Thesis Outline

This thesis is structured as follows:

- **Chapter 2: Background**
This chapter introduces key technical foundations, covering Android malware characteristics, function call graphs, Graph Neural Networks, and the GNN-LRP explanation framework. It also presents the MsDroid system, which provides the basis for subgraph extraction and model training.
- **Chapter 3: Related Work**
We review existing literature on Android malware detection techniques, graph-based app representations, GNNs in cybersecurity, and explainability in graph models. Particular attention is given to the limitations of current methods and how our approach addresses them.
- **Chapter 4: Methodology**
This chapter describes the datasets used (including a synthetic dataset and real-world Drebin and AndroZoo samples), the implementation of GNN-LRP, and improvements to the MsDroid framework. We also detail the evaluation criteria used to assess explanation quality.
- **Chapter 5: Results**
Here we present both qualitative and quantitative results from our experiments. We analyze explanation behavior on the synthetic dataset, perform a case study on the FakeInstaller malware family, and evaluate explanation robustness using a range of structural metrics.
- **Chapter 6: Conclusions and Future Work**
This chapter summarizes the key findings, highlights the contributions of the work in explainable malware detection, and outlines potential future directions, such as extending the approach to more complex graph structures or improving real-time analysis capabilities.

2 Background

This chapter introduces the key technical foundations and prior research that underpin this thesis. We begin by providing background on Android malware and the structural characteristics of APK files. We then describe how graph-based representations, such as function call graphs, are used to model app behavior. Following this, we introduce Graph Neural Networks, which offer a powerful way to process and learn from graph-structured data, and explain why they are especially relevant for malware detection. We then discuss GNN-LRP, an interpretability framework that enables detailed path-based explanations of GNN decisions. Finally, we present MsDroid, a graph-based malware detection system that serves as our primary experimental framework.

2.1 Android Malware

Android is still the most popular mobile operating system in the world, making it an attractive target for attackers [30]. Android applications are typically distributed in the form of APK (Android Package) files, which are files containing compiled code (in Dalvik bytecode format), resources, assets, and a manifest file. In this context, malware refers to any app that performs harmful or unauthorized actions, such as stealing data, spying on users, or gaining system access without permission.

2.1.1 APK Structure and Malware Characteristics

An APK file contains several components essential to understanding its behavior:

- **AndroidManifest.xml:** Declares essential app metadata, including components (activities, services), permissions, and intent filters.
- **Classes.dex:** Contains the Dalvik Executable bytecode, which includes method implementations.
- **Resources and Assets:** Media files, UI layouts, and other resources.

Malware can exploit any of these components. For example, it might request excessive permissions in the manifest, include obfuscated or suspicious code in the `classes.dex` file, or hide malicious payloads in native libraries or resources.

2.1.2 Common Malicious Behavior

Malware commonly exhibits behaviors that exploit system permissions and obfuscation techniques to evade detection. One prevalent tactic is permission abuse, where malicious applications request more permissions than are necessary for their legitimate functionality. For example, a seemingly benign application may request access to SMS messages or contacts without a valid justification. These excessive permissions can then be exploited to steal sensitive data, send premium-rate SMS messages, or track the user's location.

Another frequent technique used to avoid detection is code injection and dynamic code loading. Malware may inject malicious code into other running processes or dynamically download and execute code from a remote server at runtime. These strategies allow the

malware to alter its behavior on the fly, significantly complicating static analysis and making detection more challenging.

Malware families such as *FakeInstaller* and *DroidKungFu* demonstrate these behaviors in practice. *FakeInstaller* often masquerades as legitimate apps to trick users into granting permissions it then abuses for financial gain. *DroidKungFu* uses dynamic code loading and root exploits to gain unauthorized access to sensitive device data. These families are documented in the Drebin dataset [5], which provides a labeled collection of Android malware samples. A broader taxonomy is also available in the AMD dataset [33], which includes over 70 families with detailed annotations.

Ultimately, the primary objectives of malware are to disrupt device functionality or to exfiltrate data. Due to these evasive tactics, modern malware is often highly complex and interdependent, which makes analysis particularly difficult. Malicious behavior may be distributed across multiple functions and classes, obscuring its true intent. This is where GNNs have proven especially effective.

2.1.3 Android Malware Datasets

Several datasets have been developed to support malware research. Among the most prominent are:

- **Drebin** [5]: One of the earliest and most cited Android malware datasets (2014). It includes over 5,500 malicious APKs across more than 170 families.
- **AndroZoo** [2]: A large-scale collection of Android applications (over 20 million), both benign and malicious. It also provides rich metadata (e.g., permissions, categories) sourced from Google Play for each application.
- **AMD (Android Malware Dataset)** [33]: Released in 2017, it contains thousands of malicious samples from 71 distinct malware families. It focuses on behavior-based classification and provides granular family labels useful for fine-grained analysis and clustering.

These datasets are typically used to benchmark detection algorithms and support reproducibility in research. Malware families in these datasets exhibit varied behaviors: for instance, the *FakeInstaller* family performs premium SMS fraud, while *DroidKungFu* uses privilege escalation exploits.

2.1.4 Feature Extraction and Graph Representations

For machine learning applications, APKs need to be parsed into meaningful features. Static analysis is commonly used to extract information from the code and manifest files without running the app. Some typical features include requested permissions, sequences of API calls, intent filters and component declarations, control-flow and data-flow structures, as well as the presence of suspicious strings, such as hardcoded URLs or IP addresses.

In recent years, graph-based representations have become increasingly popular in the analysis of Android applications. One particularly useful abstraction is the function call graph, where nodes represent individual methods and edges represent the calls between them. These graphs reflect the structural behavior of an app and are well-suited for analysis with Graph Neural Networks. To apply GNN-based or even traditional machine learning models, relevant features—such as functions that access sensitive APIs—are mapped to specific nodes in the graph and properly encoded.

Figure 2.1 shows a simplified example of a call graph extracted from an Android application.

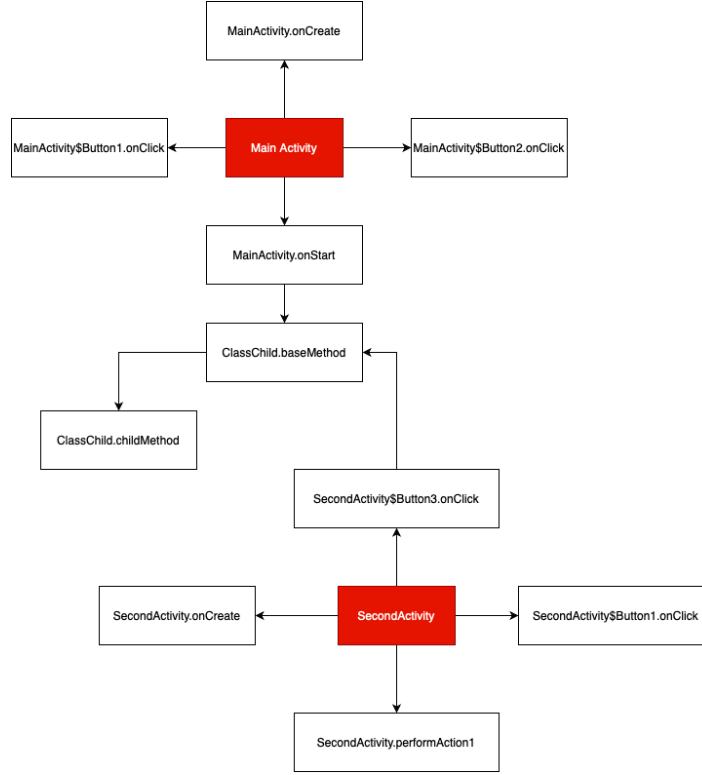


FIGURE 2.1: Example of a function call graph extracted from an Android APK. Nodes represent functions; edges represent method invocations.

Tools like MsDroid [15] (2023) use such representations to build explainable malware detection pipelines. By analyzing the structural patterns within the call graph, these models attempt to distinguish between benign and malicious logic.

2.2 Graph Neural Networks

Graph Neural Networks are a powerful class of deep learning models designed to operate on graph-structured data, where information is naturally represented as a set of entities (nodes) and relationships (edges). Unlike traditional neural networks, which are tailored to fixed-size vectors or grid-like structures (e.g., images), GNNs are capable of modeling complex, non-Euclidean domains such as molecules, social networks, and software code.

2.2.1 Fundamentals of GNNs

A graph is defined as $G = (V, E)$, where V is the set of nodes and E is the set of edges connecting them. Each node $v \in V$ may have a feature vector $\mathbf{x}_v$, and edges may also carry attributes, such as weights or types of connections.

The core idea behind GNNs is the message passing mechanism. In each layer of a GNN, nodes update their representations by aggregating information from their neighbors. A general formulation is:

$$\mathbf{h}_v^{(k)} = \text{UPDATE}^{(k)} \left(\mathbf{h}_v^{(k-1)}, \text{AGGREGATE}^{(k)} \left(\left\{ \mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v) \right\} \right) \right) \quad (2.1)$$

where $\mathbf{h}_v^{(k)}$ is the hidden state of node v at layer k , and $\mathcal{N}(v)$ is the neighborhood of v . The AGGREGATE and UPDATE functions vary depending on the specific GNN architecture.

Popular variants include Graph Convolutional Networks (GCNs) [17] and Graph Isomorphism Networks (GINs) [36].

A key consideration when working with GNNs is their computational complexity. The complexity of a GNN typically depends on the number of nodes and edges in the graph, as well as the number of message-passing iterations. In each iteration, nodes exchange information with their neighbors, and this process must be performed for all nodes and edges. Therefore, for large graphs, the computational cost can become significant. Specifically, if n is the number of nodes, e is the number of edges, and k is the number of message-passing iterations, the time complexity of a basic GNN layer is often $\mathcal{O}(k \cdot (n + e))$.

In practical applications, techniques like graph sampling or approximation methods are often employed to mitigate the computational burden of GNNs on very large graphs.

2.2.2 GNNs in Malware Detection and Cybersecurity

In cybersecurity, and especially in malware detection, GNNs are valuable because many types of malicious behavior can be naturally expressed as graphs. In the case of Android malware, this often involves building a function call graph, where nodes represent functions or classes within the APK and edges represent the call relationships between them.

These structural representations enable GNNs to learn patterns of interaction and behavior, such as frequent use of specific API sequences related to SMS sending, camera access, or network communication. They can also capture recurring graph motifs or substructures that are characteristic of certain malware families, as well as shared structural anomalies found in obfuscated or polymorphic variants.

Researchers have shown that GNN-based approaches achieve higher accuracy in malware classification compared to traditional methods that rely on flat features like permissions or hardcoded strings. Moreover, because GNNs focus on relational structures rather than fixed patterns, they tend to generalize better to new or evolving malware families.

2.2.3 Graph Isomorphism Networks

Graph Isomorphism Networks (GINs) are a type of Graph Neural Network designed to effectively distinguish between different graph structures. They rely on a sum-based aggregation of node features, followed by a multi-layer perceptron (MLP), allowing the model to capture subtle structural differences that might be overlooked by other GNN variants using mean or max aggregation.

The update rule for a node v at layer k is defined as:

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)}) \cdot \mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k-1)} \right) \quad (2.2)$$

where $\mathbf{h}_v^{(k)}$ is the node embedding at layer k , $\mathcal{N}(v)$ is the set of neighbors of v , $\epsilon^{(k)}$ is a scalar (fixed or learnable), and $\text{MLP}^{(k)}$ transforms the aggregated features.

GIN is especially useful for graph-level tasks such as Android malware classification, where small differences in call graph structure can reflect meaningful behavioral variations. In this

work, GIN is used to classify function call graphs derived from APKs, providing a strong structural foundation for downstream explanation using methods like GNN-LRP.

2.3 GNN-LRP: Interpreting Graph Neural Networks via Relevant Walks

Despite the strong predictive capabilities of GNNs, their complex internal mechanics make them inherently difficult to interpret. This limits their adoption in high-stakes domains like cybersecurity, where understanding a model’s decision is often as important as the decision itself. To bridge this gap, explanation methods such as Layer-wise Relevance Propagation (LRP) have been adapted to graph-structured data.

2.3.1 Layer-wise Relevance Propagation (LRP)

Layer-wise Relevance Propagation is a gradient-free explanation method originally developed for interpreting predictions of deep neural networks in image classification tasks [7] (2015). LRP operates by assigning a relevance score to each input feature, representing its contribution to the final prediction. These scores are propagated backward through the layers of the network from the output to the input, in a way that preserves the total prediction score:

$$f(\mathbf{x}) = \sum_i R_i \quad (2.3)$$

where $f(\mathbf{x})$ is the model’s output for input $\mathbf{x}$, and R_i is the relevance assigned to input feature x_i . Unlike gradients, which measure local sensitivity, LRP focuses on contributions, making it more robust for explaining deep models.

A key property of LRP is conservation: at each layer of the network, the total amount of relevance is preserved as it is redistributed from one layer to the next. Formally, if $R^{(k)}$ denotes the total relevance at layer k , then the conservation condition requires that

$$\sum_u R_u^{(k-1)} = \sum_v R_v^{(k)}. \quad (2.4)$$

This ensures that the explanation remains faithful by neither artificially amplifying nor diminishing the total relevance during propagation.

Formally, the backward relevance propagation at each layer involves computing the contribution of a source node u to the updated representation of a target node v . This is expressed as:

$$R_{u \rightarrow v}^{(k)} = \left(\frac{\partial h_v^{(k)}}{\partial h_u^{(k-1)}} \right) \cdot R_v^{(k)} \quad (2.5)$$

where $R_v^{(k)}$ is the relevance of node v at layer k , and the partial derivative expresses the sensitivity of v ’s representation at layer k with respect to u ’s representation at the previous layer $(k - 1)$.

2.3.2 Extending LRP to GNNs: Relevant Walks

The GNN-LRP framework proposed by Schnake et al. [29] (2022) extends LRP to GNNs by reinterpreting message passing as a series of relevant walks on the input graph. These walks are sequences of nodes and edges through which information flows and is aggregated during forward propagation. The goal of GNN-LRP is to attribute the model's output to specific walks that contributed the most to the prediction.

The explanation process in GNN-LRP begins by unfolding the computation graph of a GNN into a layered structure, capturing all message-passing operations over K layers. Relevance scores are then propagated backward layer-by-layer, not just from node to node, but from entire paths or walks that connect input nodes to the output node being explained.

In contrast to traditional LRP, where relevance is redistributed across linear layers or convolutions, GNN-LRP must account for both the aggregation of neighbor messages and the update steps in the GNN. While the formal expression for relevance propagation (Equation (2.3.1)) remains the same, its interpretation in GNN-LRP differs due to the graph structure and the message-passing mechanism. Specifically, the backward relevance propagation at each layer reflects the contribution of a source node u to the updated embedding of a target node v within the graph topology.

The final relevance assigned to an edge or a node can then be understood as an aggregation over all walks that pass through it.

2.3.3 Why GNN-LRP May Be Well-Suited for Malware Detection

We hypothesize that GNN-LRP is well-suited for the malware detection task due to its use of relevance walks, which highlight the parts of a graph that contribute most to a model's prediction. This mechanism aligns naturally with the cybersecurity domain, where Android applications are commonly represented as call graphs. These graphs represent the execution flow of a program by showing which functions invoke others. Malicious behavior often manifests as specific sequences of function calls that initiate suspicious actions, such as sending SMS messages or accessing sensitive data. In this context, relevance walks could potentially trace meaningful paths that correspond to such behaviors.

Unlike approaches that focus on individual nodes or functions in isolation, GNN-LRP captures how nodes are connected and what roles they play within the broader graph structure. We expect this to not only highlight suspicious parts of the application, but also to clarify how malicious behavior may propagate through chains of function calls. Such relational context could be particularly important in detecting more sophisticated malware.

This type of explanation may be especially valuable for security analysts, as it could offer insight into the behavioral logic of malware beyond surface-level signals. Moreover, it may mitigate the risk of misleading interpretations that can arise from noisy or ambiguous node-level indicators. Recognizing behavioral patterns in graph structures may be more robust than identifying individual malicious functions in isolation.

In this work, GNN-LRP is applied to Graph Isomorphism Network models trained on function call graphs. By visualizing the resulting relevance walks, this thesis investigates how to interpret which regions of the graph influence the model's decisions, how to identify recurring structural motifs that may be characteristic of specific malware families, and how the explanatory power of this method compares to alternative approaches.

2.4 MsDroid: Identifying Malicious Snippets in Android Apps

MsDroid is a state-of-the-art system for Android malware detection that introduces a novel approach based on graph neural networks to identify malicious code snippets within Android applications [15]. It introduces a novel subgraph-based approach to focus on specific regions of the call graph centered around sensitive APIs, rather than analyzing entire applications. This design significantly enhances scalability and allows the model to concentrate on semantically relevant behaviors.

The key motivation behind MsDroid is the observation that malicious behaviors in Android malware are typically implemented as small, localized code regions surrounding sensitive APIs. Traditional whole-app analysis often results in redundant or noisy feature representations, which can degrade the performance of classifiers. MsDroid addresses these challenges by mimicking the workflow of human analysts: it isolates and focuses on local behavior around sensitive APIs.

A significant challenge in applying GNNs to Android malware detection lies in their computational complexity, which increases with the size of the input graph. This can become a bottleneck when analyzing large or even moderately sized applications (see Section 2.2.1). MsDroid addresses this issue by operating on smaller, context-relevant subgraphs centered around sensitive APIs. This not only improves computational efficiency but also enhances detection performance and makes the resulting explanations more transparent to users.

MsDroid achieves robust and interpretable detection through the use of a graph-based representation that captures both code structure and domain knowledge. Its performance is demonstrated across 81,790 apps from three major malware datasets (Drebin, AMD, and Androzoo), where it outperforms baseline models by up to 49.52% in F1-score under challenging scenarios such as zero-day threats, code obfuscation, and app evolution.

2.4.1 Adopted Components in This Thesis

In this thesis, we incorporate two core components of MsDroid into our experimental pipeline:

- **Behavioral Subgraph Generation:** We adopt MsDroid’s subgraph generation strategy to create graph-structured representations of potentially malicious code regions.
- **Graph Neural Network Models:** We employ the GNN architectures used in MsDroid for training and evaluating our own malware detection models.

2.4.2 Subgraph Generation Methodology

MsDroid transforms the static call graph of an Android application into a set of subgraphs centered around sensitive APIs. This set is referred to as the *Behavior Subgraph Set*. The subgraph generation proceeds as follows:

1. **Call Graph Extraction:** Static analysis is performed using Androguard to extract the full call graph $G = (V, E)$, where nodes represent APIs and edges denote invocation relationships.
2. **Sensitive API Identification:** Nodes corresponding to permission-sensitive APIs are identified using mappings from PScout [6] (2012) and Axplorer [8] (2016).
3. **Subgraph Induction:** For each sensitive API node $v_i \in V_{\text{per}}$, a subgraph is induced over its k -hop neighborhood $N^k(v_i)$. Nodes are categorized into internal user-defined APIs, third-party library APIs, and external system APIs.

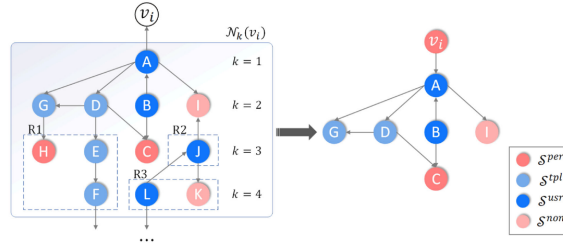


FIGURE 2.2: MsDroid subgraph generation from [15]

4. **Node Reduction:** Non-informative or redundant nodes are pruned based on three rules:
 - Third-party and external APIs not directly invoked by user-defined code are discarded.
 - User-defined nodes that do not reach the sensitive API are excluded.
 - Isolated nodes with no path to the sensitive API are removed.
5. **Feature Construction:** Each node's feature vector combines opcode frequency vectors (for internal nodes) and permission vectors (for sensitive APIs), including propagation of permission information through TPL nodes when necessary.

2.4.3 GNN Models for Subgraph Classification

MsDroid employs a message-passing GNN with a multilayer perceptron for node embedding updates. Node features are updated over three aggregation steps, and the final subgraph representation is computed using a combination of max-pooling and mean-pooling. The GNN is trained with a custom *app subgraph loss* function, which accommodates the lack of fine-grained snippet labels:

$$\mathcal{L} = (1 - y) \frac{1}{|G|} \sum_{g \in G} \log(1 - p_g) + y \min_{g \in G} \log(p_g) \quad (2.6)$$

where p_g is the predicted probability that subgraph g is malicious, and $y \in \{0, 1\}$ is the app-level label.

This formulation ensures that at least one subgraph in a malware app is detected as malicious, while benign apps are expected to have no malicious subgraphs.

2.4.4 Evaluation and Results

MsDroid demonstrates impressive detection performance, achieving a 97.82% testing accuracy and outperforming state-of-the-art systems by large margins in terms of robustness against zero-day samples, obfuscation, and app evolution.

2.4.5 Explanation Mechanism

MsDroid introduces a three-level explanation mechanism to enhance the interpretability of its detection:

- **Suspicious API Usage:** Highlights sensitive APIs implicated in malicious behaviors.

- **Calling Heat Graphs:** Visualizes control-flow dependencies with edge importance scores derived from learned attention masks.
- **Similar Behavioral Snippets:** Performs embedding-based similarity search against known malware to aid analysts in understanding new threats.

Chapter Summary

This chapter introduced the key concepts and motivations for using explainable Graph Neural Networks in Android malware detection. We showed how Layer-wise Relevance Propagation can be adopted to graph-structured data with GNN-LRP, producing explainable relevance walks that point out the malicious behavior.

We also presented MsDroid, the underlying framework that enables scalable and targeted analysis by extracting subgraphs centered around permission-requiring APIs. Together, these components form the foundation of this work and support the central hypothesis of the thesis: that GNN-LRP can provide efficient, interpretable explanations for malware classification decisions.

The next chapter presents the methodological approach used to apply and validate this technique.

3 Related Work

This chapter reviews the existing literature relevant to Android malware detection, graph-based representations of Android applications, the use of Graph Neural Networks in this domain, and the growing field of explainable artificial intelligence (XAI), particularly in security-sensitive contexts. Emphasis is placed on techniques that align closely with our approach, which combines GNN-based learning with explainability using GNN-LRP and graph representations generated via the MS-Droid framework.

3.1 Android Malware Detection Techniques

There are two main techniques for performing malware analysis.

Static analysis involves examining an application’s code and associated resources without executing the program. This method enables the extraction of features such as permissions, API calls, and control flow structures. The advantages of static analysis include its efficiency and the ability to analyze a large number of applications quickly. However, it usually struggles with obfuscated code and cannot capture behaviors that manifest only during runtime [16].

Dynamic analysis, in contrast, observes the behavior of applications during execution, typically within a controlled environment or sandbox. This approach can detect malicious activities that are not evident in static code, such as dynamic code loading or runtime permission requests. While dynamic analysis provides a more comprehensive view of an application’s behavior, it is resource-intensive and may not achieve complete code coverage, potentially missing some malicious actions [16].

In the realm of static analysis, machine learning and deep learning techniques have been employed to enhance malware detection. Notable models include Drebin, which uses a linear SVM classifier on features extracted from the manifest and code [5]; MaMaDroid, which models sequences of API calls as Markov chains to detect anomalous behavior [22]; and MsDroid, which leverages Graph Neural Networks to identify malicious patterns in the application’s graph representation [15]. These models have demonstrated high accuracy in detecting known malware and show promise in identifying previously unseen threats.

3.2 Graph-Based Representations for Android Applications

Graph-based representations are central to the static analysis of Android applications, as they enable the modeling of complex program behaviors in a structured and analyzable form. Several types of graphs are commonly employed to capture different aspects of an application. Control Flow Graphs (CFGs) are used to represent the flow of execution within individual methods, illustrating all possible paths that the control can take. Inter-Component Call Graphs (ICCGs), on the other hand, model interactions between different components of an Android application, such as Activities, Services, Broadcast Receivers, and Content

Providers. These interactions are crucial to capture due to the component-based architecture of the Android platform. Abstract Control Graphs (ACGs) provide a higher-level view by combining control flow and data dependencies, often aggregating multiple methods and components into a unified graph that abstracts over low-level details while preserving important structural information.

To build these representations, a reliable call graph of the application must first be extracted. This is typically done by decompiling the Android Package (APK) and analyzing its code and metadata. Tools such as Androguard and APKTool are widely used in the community for this purpose. APKTool facilitates the decoding of resources and manifests, allowing for an understanding of the application's declared components and permissions. Androguard performs bytecode analysis and is capable of constructing detailed call graphs by examining method invocations and class hierarchies in the Dalvik bytecode [3, 4]. These tools form the foundation for constructing the graph-based inputs needed for subsequent static analysis.

Creating a precise call graph is not only necessary for downstream learning models to perform well, but also for ensuring that the analysis is sound. The soundness of a static analysis refers to its ability to over-approximate all possible program behaviors in a conservative manner. A recent study by Samhi et al. [28] examined the soundness of several state-of-the-art static analysis tools in the Android ecosystem and found that, on average, these tools missed over 60% of methods that were actually executed at runtime. This finding highlights the challenges in accurately modeling dynamic features in Android, such as reflection, dynamic code loading, and complex lifecycle behaviors. Inaccurate or incomplete call graphs lead to blind spots in the analysis, which in turn can undermine both the correctness and the security guarantees that the analysis aims to provide.

3.3 Graph Neural Networks in Malware Detection

Graph Neural Networks have emerged as powerful tools for learning over structured data, including graphs that model program behavior. In the context of Android malware detection, GNNs are particularly effective due to their ability to capture both syntactic and semantic relationships by aggregating information from node neighborhoods [34]. This makes them well-suited for analyzing graph representations such as Control Flow Graphs, Call Graphs, and Inter-Component Communication Graphs that encode the structural and behavioral properties of applications.

Among the most prominent approaches is MsDroid, which analyzes semantic subgraphs of application call graphs using a GNN model to identify malicious code snippets. This approach not only yields high detection accuracy, but also offers interpretability by linking predictions to specific graph regions, helping analysts understand model decisions [15]. GraphDroid further exemplifies the power of GNNs by applying them over Inter-Component Call Graphs (ICCGs), capturing contextual relationships between program entities to effectively distinguish between benign and malicious applications [31]. Other frameworks such as GSEDroid and GNNDroid have extended this paradigm by incorporating heterogeneous data sources—like opcode sequences, API usage, and permissions—into enriched graph structures, enabling a more comprehensive view of application behavior [14, 35].

Collectively, these GNN-based approaches represent the current state-of-the-art in Android malware detection using static analysis. They provide both improved predictive performance and valuable insight into the structure and behavior of potentially malicious applications.

3.4 Explainable AI in Security and Malware Analysis

Explainable Artificial Intelligence (XAI) has become increasingly important in the field of Android malware detection, where understanding the rationale behind model predictions is essential for trust, validation, and effective security response.

A prominent example of explainability in this domain is MsDroid, a GNN-based system designed to identify malicious code snippets by analyzing semantic subgraphs within application call graphs. Beyond achieving high detection accuracy, MsDroid provides interpretability by linking its predictions to specific regions of the graph. This feature helps security analysts understand why a sample is classified as malicious, offering greater transparency and support for manual investigation [15].

Underscoring the necessity of explainable approaches, Liu et al. [27] conducted a comprehensive study that applied XAI techniques to audit machine learning-based Android malware detectors. Their analysis revealed that certain high-performing models obtained strong results by unintentionally learning from temporal artifacts in the training data, rather than from meaningful malicious behavior. This insight reinforces the importance of explainability not only for transparency but also for ensuring that models are robust and generalize well to real-world threats.

3.5 GNN Explainability: Techniques and Applications

Explainability is critical when deploying Graph Neural Networks in security-sensitive domains like malware detection. One of the earliest approaches, GNNExplainer, identifies a minimal subgraph and feature subset responsible for a model's prediction. While effective in some settings, it often produces unstable and inconsistent results, limiting its reliability in high-stakes applications such as security analysis.

Addressing these limitations, MsDroid introduces a domain-specific explanation mechanism for Android malware detection. Instead of analyzing entire application graphs, it decomposes apps into behavior subgraphs centered around sensitive APIs. Predictions are made at the subgraph level, enabling more localized and interpretable insights. Detected malicious subgraphs are visualized using a calling heat graph that highlights important control-flow nodes, and similar known malicious patterns are retrieved to contextualize novel threats [15].

Building on the need for more principled approaches, GNN-LRP extends the Layer-wise Relevance Propagation technique to GNNs, offering theoretically grounded, faithful explanations. It propagates relevance scores across graph layers, making it potentially very well-suited for domains like malware analysis where traceable and robust justifications are essential [29].

Chapter Summary

Although significant advancements have been made in applying GNNs to malware detection and XAI, there is still a critical need for interpretable and strong graph-based detection systems. The majority of existing GNN models lack build-in mechanisms for interpretability, and conventional XAI methods struggle to provide valuable insights when handling graph-structured data.

Our work addresses this gap by integrating GNN-LRP with the MsDroid framework to extract semantically meaningful subgraphs and build strong Android malware detection models.

4 Methodology

4.1 Datasets

Three main datasets were used for the experimental component of this thesis: a custom synthetic dataset, the Drebin dataset, and the Androzoo dataset. The Drebin dataset provides labeled Android malware samples, while benign applications were sourced from the Androzoo corpus. These were used to construct a balanced and graph-structured dataset suitable for evaluating the GNN-LRP explanation framework in malware detection.

4.1.1 Synthetic Dataset

To validate the GNN-LRP explanation method, we initially evaluated it using a custom-designed toy dataset. This dataset consists of 10 small and randomly selected benign Android applications sourced from AndroZoo. We specifically choose smaller applications to allow for manual inspection and a detailed analysis. AndroZoo was selected because of its accessibility, extensive metadata, and widespread adoption in prior research, including MsDroid.

To simulate malware behavior in a controlled setting, a manually crafted malicious payload was injected into each APK. From these modified applications, we extracted subgraphs derived from their call graphs, resulting in a total of 49 subgraphs: 29 labeled as malicious and 20 as benign.

The base benign applications used in this synthetic dataset are listed in Appendix B.1.

The injected payload implements a simplified malware behavior pattern. As shown in Figure 4.1, the injected call graph includes the functions `gatherDeviceInfo`, `connectToServer`, and `prepareMaliciousPayload`.

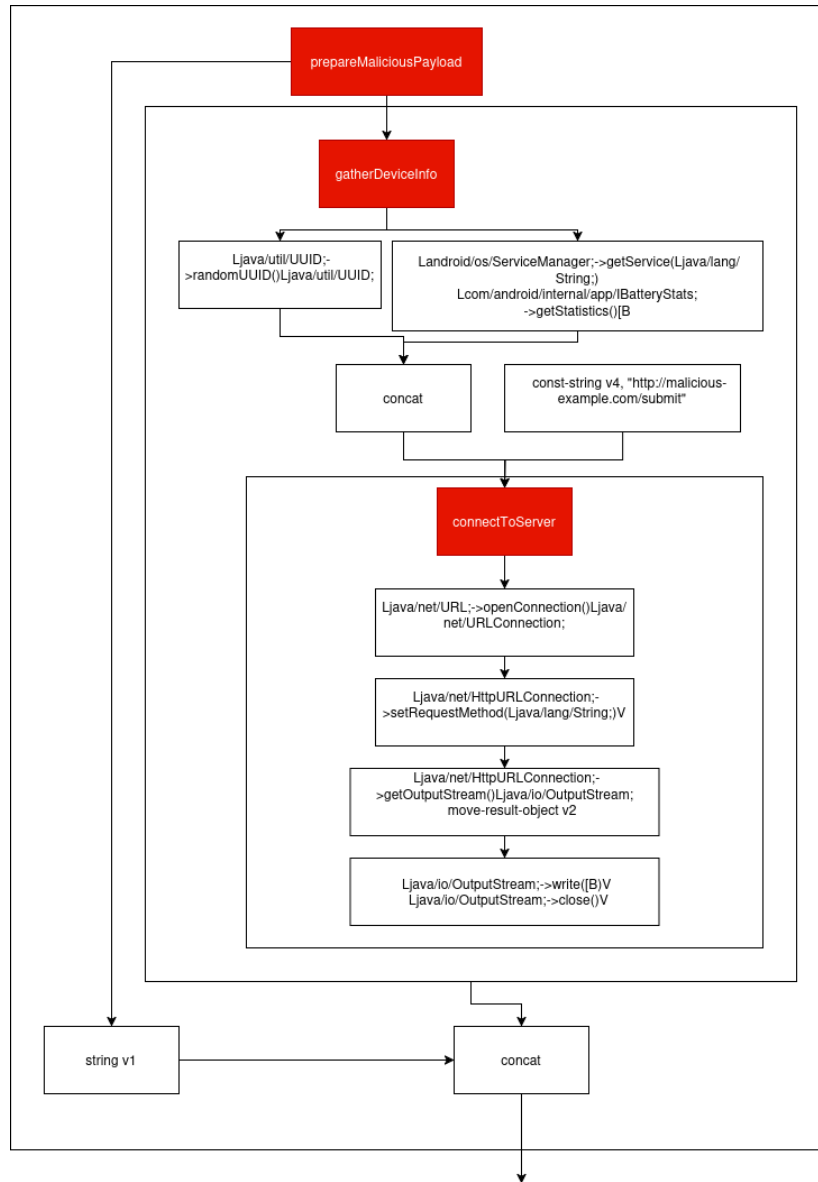


FIGURE 4.1: Simplified call graph representing the injected malware behavior.

To generate benign counterparts, the same structure was reused with one key modification: the function `connectToServer` was omitted. This function was specifically excluded because it represents the final step in the malicious behavior chain and directly signals external communication, a common characteristic of malware. By omitting this function from the benign training examples, the model should learn that a subgraph is malicious if and only if it contains this function.

This setup is also intended to test whether GNN-LRP can accurately detect the influence path across multiple layers. Ideally, although this strongly depends on what the model learns to focus on during training, GNN-LRP should be able to highlight the walk that corresponds to the full malicious call sequence: `prepareMaliciousPayload` → `gatherDeviceInfo` → `connectToServer`. This is further explored in Section 5.1.

4.1.2 Drebin Dataset

The Drebin dataset [5] is a widely used benchmark for Android malware detection. It includes thousands of labeled malicious Android applications across numerous malware families. Specifically, Drebin categorizes malware into 179 different families, each representing a distinct type of malicious behavior such as data exfiltration, SMS fraud, adware, remote access tools, and privilege escalation attacks. These families serve to capture the diversity of the Android malware ecosystem and provide a representative set of threats for evaluating detection techniques. The applications in Drebin were processed using the MsDroid tool to extract call graphs and generate behavior subgraphs.

Due to challenges during graph generation—such as corrupted APKs, missing bytecode, or absence of sensitive APIs—some samples were excluded, resulting in a slightly smaller dataset than the original Drebin corpus. Detailed statistics about subgraphs and APKs per malware family are provided in Tables A.1–A.4, which appear at the end of this section. Overall, Drebin contributes 4851 malicious APKs and 73,361 behavior subgraphs, with an average of 676 nodes and 4442 edges per subgraph, as summarized in Table 4.1.

For our evaluation of explainability, we begin with an in-depth analysis of the FakeInstaller family, one of the largest and most well-known malware families in the Drebin dataset. This family is particularly relevant for our study for several reasons. First, its large number of samples provides a statistically meaningful basis for evaluating explanation consistency and detection robustness. Second, FakeInstaller represents a class of SMS-based malware that relies on sending premium-rate text messages without user consent. Such behavior often involves clearly defined control flows and sensitive API calls—such as `sendTextMessage()`—making the malicious functionality more easily identifiable through both structural analysis and visualization.

4.1.3 Androzoo Dataset

Androzoo [2, 1] is a comprehensive collection of Android applications from various sources, including both benign and malicious samples. For this thesis, benign applications were extracted from Androzoo, using specific filtering criteria to ensure suitability and diversity. The corpus currently contains over 21 million APKs. The average APK file size in our subset is 21.71 MB. Although originally published in 2016, Androzoo continues to incorporate modern applications, providing a broad temporal range.

From this dataset, we selected 3373 benign APKs, which were successfully processed to generate 73649 subgraphs, as summarized in Table 4.1.

4.1.4 Dataset Summary

This section provides a concise overview of the key characteristics of each dataset used in this study.

TABLE 4.1: Dataset statistics. *APK* = total APKs, *M-APK* = malicious APKs, *SG* = total subgraphs, *M-SG* = malicious subgraphs, *N/S* = avg. nodes per subgraph, *E/S* = avg. edges per subgraph, *Deg.* = avg. degree, *Dens.* = avg. density, *Size* = avg. file size (MB).

Dataset	APKs	M-APK	SG	M-SG	N/S	E/S	Deg.	Dens.	Size
Toy Dataset	20	10	49	29	14.50	18.10	1.26	0.21	0.031
Drebin	4851	4851	73361	73361	676	4442	5.73	0.072	1.42
Benign Dataset	3373	0	73649	0	746	2932	3.57	0.0635	1.54

As mentioned above, synthetic dataset serves as a baseline for validating the GNN-LRP explanation method. Its small size and low complexity allow for detailed manual inspection of the generated explanations. In contrast, the Drebin and Androzoo datasets represent real-world Android applications and thus feature significantly greater scale and complexity. The approach for experimentation begins with the toy dataset, followed by analysis of a single family from Drebin, and then expanding to examine a specific type of malware within Drebin—such as all families related to SMS malware.

4.2 GNN-LRP Implementation

4.2.1 Overview

GNN-LRP (Graph Neural Network Layer-wise Relevance Propagation) is an interpretability framework designed to attribute a model’s prediction to the graph elements that influenced it most. It extends the concept of relevance propagation to the graph domain, where relational structure and message-passing mechanisms play a central role in the model’s behavior.

The key idea behind GNN-LRP is to propagate a model’s output score backward through the layers of a Graph Neural Network, redistributing it across nodes and edges based on their contribution to the prediction. This is achieved using custom backpropagation rules adapted for different GNN layer types, such as message-passing, linear, and pooling layers.

Our implementation generalizes the GNN-LRP framework for arbitrary PyTorch Geometric-based architectures. It departs from the original demonstration code from the paper, by supporting fully modular relevance propagation, capturing intermediate activations via forward hooks and applying custom rules during the backward pass.

In addition, we introduce a walk-based relevance filtering mechanism. This enables relevance to be routed selectively along predefined node paths, allowing for path-specific interpretability and fine-grained control over how decision-making flows are visualized in large graphs.

This architecture-agnostic design ensures compatibility with a wide range of GNN models while minimizing the need for manual intervention. The following sections detail the internal logic, propagation rules, and technical implementation that make this possible.

4.2.2 Class Design and Implementation

The main class, `GNNLRP`, wraps any PyTorch Geometric-based GNN. It automatically detects layer types and attaches hooks that capture the inputs and outputs during inference. During relevance propagation, it applies the corresponding rule for each module in reverse order.

A relevance conservation check is performed at each layer to ensure correctness:

$$\sum_i R_i^{(l+1)} = \sum_j R_j^{(l)} \quad (4.1)$$

If a predefined walk is provided, relevance is selectively routed along the specified node paths, which is useful for tracing influence in large graphs.

4.2.3 Message Passing Relevance Rule

Message-passing layers form the core of GNN architectures. Here, we implement a relevance propagation rule that matches the GIN aggregation mechanism:

$$h'_i = (1 + \epsilon) \cdot h_i + \sum_{j \in \mathcal{N}(i)} h_j \quad (4.2)$$

During backpropagation, each node's relevance is redistributed proportionally to its own contribution and those of its neighbors:

$$R_i = \frac{(1 + \epsilon)h_i}{(1 + \epsilon)h_i + \sum_{k \in \mathcal{N}(i)} h_k + \epsilon} \cdot R_i \quad (4.3)$$

Code Summary: The function below implements this rule by computing how much relevance to attribute to each node and its neighbors. If an explanation walk is provided, it restricts propagation to the selected path for interpretability.

```
def message_passing_lrp_rule(self, layer, R, epsilon=1e-9):
    Z = layer.input
    mp = self.map_connections(layer.edge_index)
    R_prime = torch.zeros_like(R)

    for src, destinations in mp.items():
        if self.walk is None or self.walk[self.walk_idx][0] == src:
            denominator = (1 + layer.eps) * Z[src] +
                Z[destinations].sum(dim=0) + epsilon

            for i, destination in enumerate(destinations):
                if self.walk is None or self.walk[self.walk_idx][1]
                    == destination:
                    contribution = Z[destination] / denominator *
                        R[src]
                    R_prime[destination] += contribution

            if self.walk is None or self.walk[self.walk_idx][1] ==
                src:
                contribution = ((1 + layer.eps) * Z[src]) /
                    denominator * R[src]
                R_prime[src] += contribution

    if self.walk is not None: self.walk_idx += 1
    return R_prime
```

LISTING 4.1: Message-passing LRP with walk filtering

4.2.4 Linear Layers

Fully connected (linear) layers follow a traditional LRP formulation. Each input feature contributes to the output based on the weighted sum of its activations. Relevance is redistributed to each input according to this contribution:

$$R_i = \sum_j \frac{x_i w_{ij}}{\sum_k x_k w_{kj} + \epsilon} R_j \quad (4.4)$$

Code Summary: This function computes the proportion of each input feature’s contribution to the output, then distributes relevance accordingly. It supports layers with arbitrary shapes and batch sizes.

```
def fc_lrp_rule(layer, R, epsilon=1e-9):
    Z = layer.input
    W = layer.weight
    S = layer.output

    numer = torch.einsum('bi,oi->boi', Z, W)
    denom = torch.einsum('bi,oi->boi', Z, W).sum(dim=2, keepdim=True)
            + epsilon
    relevance_contrib = numer / denom + epsilon

    R_expanded = R.unsqueeze(-1)
    relevance = (relevance_contrib * R_expanded).sum(dim=1)
    return relevance
```

LISTING 4.2: Linear layer LRP

4.2.5 Pooling Layers

Pooling layers summarize node-level features into a global graph representation. During LRP, relevance must be redistributed back from the global summary to the individual nodes. Different pooling strategies require different propagation rules:

Max Pooling: Only the node(s) with the maximum activation receive relevance:

$$R_i = \mathbb{1}_{z_i=\max_j z_j} \cdot \frac{R}{\sum \mathbb{1}} \quad (4.5)$$

Mean Pooling: Relevance is evenly distributed across all nodes:

$$R_i = \frac{R}{n} \quad (4.6)$$

Mixed Pooling: A weighted combination of max and mean pooling relevance:

$$R = \alpha R_{\max} + (1 - \alpha) R_{\text{mean}} \quad (4.7)$$

4.2.6 GNN-LRP Visualization

To better interpret the relevance computed by the GNN-LRP explanation method, we design three types of visualizations: (i) walk-level relevance plots, (ii) edge-level relevance plots,

and (iii) node-level relevance plots. Each of these provides a different perspective on how the model attributes importance across the graph structure.

The walk-level relevance plot provides the most direct representation of the GNN-LRP output. Since GNN-LRP assigns relevance scores to walks — sequences of connected edges with a length equal to the number of message-passing layers in the model — this plot visualizes all such walks within the subgraph. However, these visualizations can become dense and difficult to interpret, particularly in large subgraphs or when relevance is spread across many different walks. To address this, we complement the walk-level plots with edge- and node-level summaries. While these aggregated views are less faithful to the model’s internal representation, they can help users more easily identify the most relevant regions of the subgraph.

The edge-level relevance plot aggregates the relevance across all walks in which a given edge appears. Since the same edge can be part of multiple walks, we compute the average relevance per edge to obtain a more stable and interpretable score. This is done by summing the relevance contributions from each walk containing the edge and dividing by the number of times the edge appears across all walks. Averaging in this way ensures that edges frequently included in high-relevance walks are emphasized, while preventing over-representation of edges that simply appear in many walks with low individual relevance. Without averaging, the relevance plot would be biased toward frequently occurring edges regardless of their actual importance.

Similarly, the node-level relevance plot provides a summary by aggregating relevance across all walks in which a node participates. For each walk, the relevance score is attributed equally to the nodes involved in the corresponding edges, and the final node relevance is again averaged over the number of times each node appears in the walks. As with edges, this normalization ensures a fair representation of each node’s overall contribution to the model’s decision, independent of how many times the node appears in various walks. Without this step, nodes that occur frequently in the graph structure, regardless of their actual influence, might dominate the visualization.

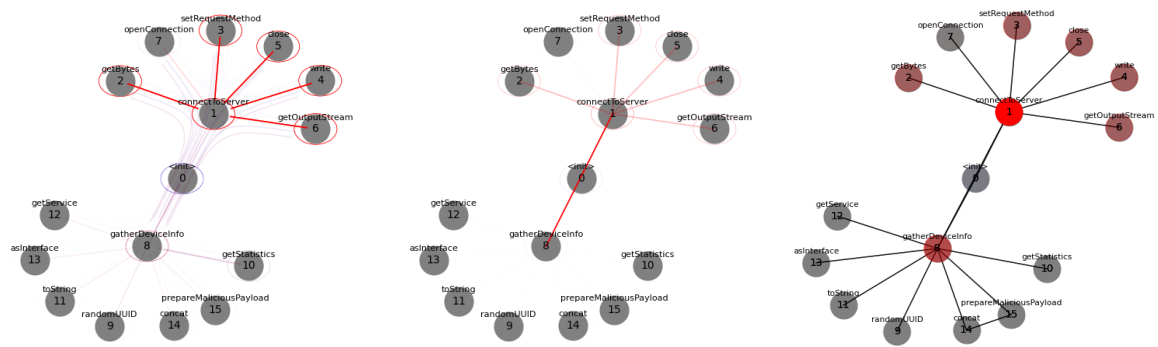


FIGURE 4.2: GNN-LRP visualizations on a sample subgraph. Left: walk-level relevance plot Center: edge-level relevance summary. Right: node-level relevance summary.

4.3 MsDroid Subgraph Creation

In this section, we present the improvements made to MsDroid regarding the creation of subgraphs. Previously, MsDroid had a key limitation: it could not accurately represent

recursive calls or connections that extended beyond a simple call chain. When a function already included in the subgraph was called again, the system created a new node instead of referencing the existing one. This approach led to a fragmented view of the call graph, making it difficult to identify recurring behavior and long hierarchical patterns.

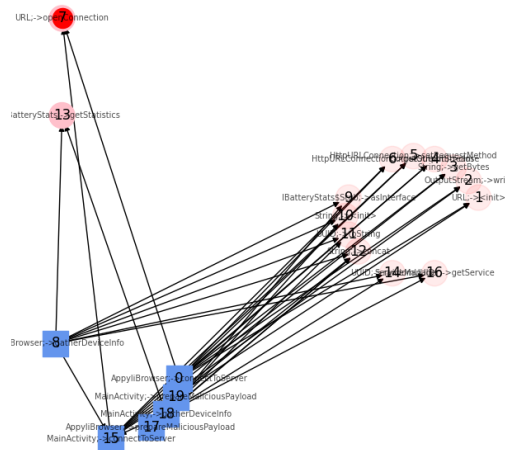


FIGURE 4.3: Example of a subgraph in the original MsDroid representation

To overcome this, we introduced a mechanism that allows nodes in the subgraph to reference existing nodes already explored in the graph. This change enables the representation of recursive calls and the expression of higher-order paths between functions. With this new design, if a function is called multiple times, it is represented as a single node in the subgraph, with all relevant edges properly connected.

This approach significantly improves the graph’s ability to highlight meaningful relationships. This change becomes more significant the larger we set the HOP variable to.

For instance, imagine a scenario where a benign function sends a POST request to a server. Later in the graph, a malicious function makes the same type of call, but to a malicious domain. In the old model, these would appear as entirely separate nodes, making it difficult for the model—or any explanation method—to trace the connection between the two. With the new representation, these calls are unified, and their contextual differences are preserved as metadata, allowing for a much clearer interpretation.

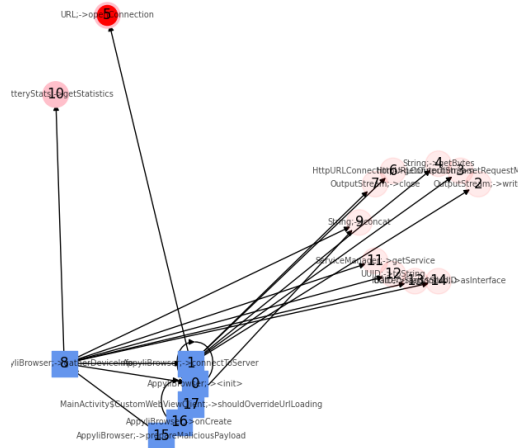


FIGURE 4.4: Improved subgraph showing merged nodes and recursive paths

We can observe an example of this by comparing the images provided. In Figure 4.3, several nodes—such as `gatherDeviceInfo`, `connectToServer`, and `maliciousPayload`—are unnecessarily duplicated. This redundancy reduces the effectiveness of the explanation method, especially when dealing with large and complex subgraphs. By unifying these repeated nodes into a single representation, we not only reduce visual clutter but also preserve the structural integrity of the graph, making it easier to trace functional patterns and understand behavior across different parts of the application.

4.4 Evaluation Framework

To rigorously assess the quality and reliability of the GNN-LRP explanation method in the context of malware detection with graph neural networks, we adopt and adapt the Co-12 framework proposed by Nauta et al. [24]. This framework outlines twelve desirable properties of explanation methods. Given our setting and goals, we focus on a tailored subset of these properties that are both measurable in our experimental setup and relevant to the malware analysis domain. Our evaluation emphasizes functionally-grounded, model-faithful explanations, primarily assessed using a synthetic dataset where the ground-truth causal structure is known.

4.4.1 Correctness

Correctness refers to the faithfulness of the explanation with respect to the actual decision process of the model. A correct explanation should highlight input components that truly influenced the prediction. To evaluate this, we implement several tests:

Model Parameter Randomization. We randomize the weights of a trained GNN and apply GNN-LRP to generate explanations. If the method is faithful, the output should be unstructured and semantically meaningless, as no real knowledge was learned by the model.

Label Randomization. The model is trained from scratch on a version of the dataset with randomized class labels. Explanations in this case should again be incoherent or biased toward structural artifacts, revealing that meaningful structure only emerges when supervision is correct.

Ground-Truth Recovery in Synthetic Data. On the synthetic dataset—where malicious behavior is injected via specific function chains—we assess whether the explanation correctly identifies the known malicious substructures. In particular, we examine whether GNN-LRP recovers the injected chain involving `connectToServer`, `gatherDeviceInfo`, and `prepareMaliciousPayload`.

These three tests evaluate the model’s ability to produce semantically aligned explanations when learning is valid, and to degrade gracefully when it is not.

4.4.2 Completeness

Completeness measures how fully the explanation captures the model’s decision logic. An explanation is considered complete if it accounts for most of the prediction-relevant information.

We evaluate completeness using deletion-based strategies:

Incremental Deletion. We remove edges from the input graph in order of decreasing relevance score and monitor the model’s confidence or predicted class at each step. A sharp decline in confidence indicates that the explanation identifies decision-critical components.

Preservation and Deletion Duality. This test verifies both preservation and removal behavior. If only the top- k most relevant edges are retained, the model should still make the same prediction. Conversely, removing those top- k edges should lead to a confidence drop or misclassification. We use this duality to assess whether the explanation captures enough of the input to reproduce or change the prediction.

Fidelity Curves. We quantify prediction fidelity as a function of the fraction of edges retained or removed. These curves offer a more global view of how well explanations align with the model’s predictive behavior.

4.4.3 Contrastivity

Contrastivity ensures that explanations are sensitive to what is being predicted. That is, explanations for different outputs—e.g., malware vs. benign—should differ meaningfully, reflecting different discriminative evidence.

We evaluate this in two ways:

Target Sensitivity. We compare explanations between malware and benign samples in the synthetic dataset. We expect explanations for malware predictions to highlight injected malicious chains, and benign explanations to avoid such regions.

Supervision Sensitivity. As in the correctness tests, we use label-randomized training to examine whether contrastive structure disappears when meaningful class boundaries are removed.

4.4.4 Continuity

Continuity captures explanation stability under small perturbations to the input. If two inputs are nearly identical and their predictions do not change, their explanations should also be similar.

We measure continuity qualitatively and functionally:

Perturbation Stability. We introduce small, non-disruptive changes to the graph—such as removing peripheral nodes or edges—and examine whether the explanation remains stable in key regions.

Functional Consistency. We check that explanations derived from the original input still hold for perturbed versions, provided the model’s output does not change.

Continuity is especially important for robustness, ensuring that explanations do not fluctuate erratically with minor graph changes.

4.4.5 Consistency

Consistency refers to the reproducibility of explanations across different versions of the model trained on the same task. Even when trained with different hyperparameters or initializations, a reliable explanation method should highlight the same important features if the model predictions are the same.

We test consistency through:

Training Variation. We retrain the GNN with variations in batch size, training epochs, and network depth (e.g., using 3 vs. 4 message-passing layers). We then compare the resulting explanations on key malicious samples.

We expect the same injected behavioral structure to be identified across runs, showing that GNN-LRP explanations are stable with respect to training noise or architecture choice.

4.4.6 Compactness

Compactness evaluates whether explanations are concise and focused. A good explanation should highlight a minimal subset of the input graph that still suffices to explain the decision.

We assess compactness using two metrics:

Sparsity. We compute the fraction of nodes and edges with non-zero relevance in each explanation. Lower values indicate more compact explanations.

Redundancy Entropy. This measures how diverse the explanation is. Higher entropy implies that relevance is spread across structurally different components, indicating a rich, non-repetitive explanation. Low entropy may suggest redundancy or overemphasis on a small motif.

These metrics are computed across multiple synthetic samples to evaluate structural parsimony.

4.4.7 Coherence

Coherence ensures that the explanation aligns with domain knowledge—in this case, malware analysis practices in Android applications.

We evaluate coherence qualitatively by manually inspecting whether explanations highlight semantically suspicious behavior, such as API calls related to network access (`connectToServer`), device profiling (`getStatistics`, `gatherDeviceInfo`), or payload preparation.

Since we explicitly define the malicious behavior in the synthetic dataset, this test is particularly meaningful: we know in advance what should be highlighted, and coherence is measured by alignment with that expected structure.

In summary, our evaluation framework systematically tests seven key properties of explanation quality: correctness, completeness, contrastivity, continuity, consistency, compactness, and coherence. The synthetic dataset enables precise testing through controlled ground-truth structures, while graph-level deletion and perturbation tests allow us to quantitatively assess model faithfulness. This multifaceted approach ensures that GNN-LRP is evaluated not only for visual interpretability, but also for robustness, stability, and alignment with the model’s internal reasoning.

Chapter Summary

In this chapter, we described the methodology used to evaluate GNN-LRP for explainability in graph-based Android malware detection. We introduced the three core datasets used in our experiments: a custom synthetic dataset designed for ground-truth validation, the Drebin dataset for malware samples, and benign samples sourced from Androzoo.

We then detailed the implementation of the GNN-LRP explanation method, which supports any GNN architecture as long as the base LRP rules for each type of layer are defined. We also developed multiple relevance visualization strategies—at the node, edge, and walk levels—to support both quantitative and qualitative assessments.

To improve the expressiveness and correctness of graph construction, we made several modifications to the MsDroid framework. Specifically, we improved subgraph generation to correctly handle recursive calls without recreating nodes. This results in cleaner and semantically more meaningful graphs, which benefit both the learning process of the model and the accuracy of user interpretations.

Finally, we presented a comprehensive evaluation framework, adapted from the Co-12 protocol, to assess the quality of GNN-LRP explanations. We focused on seven essential properties—correctness, completeness, contrastivity, continuity, consistency, compactness, and coherence—and employed a combination of sanity checks, graph perturbations, deletion-based analysis, and known malicious behaviors to evaluate them. This provides a rigorous foundation for assessing explanation quality in the context of graph-based malware detection.

5 Results

5.1 Synthetic Dataset

To validate the correctness of our GNN-LRP implementation, we first conducted experiments on a synthetic dataset derived from benign applications sourced from AndroZoo. Malicious samples were artificially constructed by injecting code snippets designed to simulate known malicious behaviors, as described in Section 4.1.1.

This controlled setup allows us to evaluate the explanation model in a context where the malicious behavior is explicitly defined. In such an environment, we can precisely assess whether the explanations produced by the model align with the intended injected logic.

We begin by training a MsDroid GNN model on this dataset and apply its built-in explanation mechanism to examine which parts of the graph it identifies as malicious. We then apply GNN-LRP to the same samples and analyze the subgraph regions it highlights. The outputs of both methods are compared in terms of interpretability, specificity, and their alignment with the ground-truth injection patterns.

It is important to emphasize that explanation methods reflect the model’s internal decision logic, which may not always appear intuitive to a human observer. An explanation might seem unexpected but still faithfully represent the decision boundary learned during training. However, given the known injection pattern, we expect that GNN-LRP should attribute relevance to the key malicious component—specifically, the `ConnectToServer` node and, ideally, its related call chain: `connectToServer` $\rightarrow$ `gatherDeviceInfo` $\rightarrow$ `prepareMaliciousPayload`.

Finally, we apply the proposed evaluation framework to this synthetic dataset to produce a quantitative assessment of explanation quality and robustness under controlled conditions.

5.1.1 Experimental Setup

For this experiment, we trained the MsDroid GNN model using the following configuration: three message-passing layers, a dropout rate of 0.25, a batch size of 16, a training split of 70%, max-based global pooling, and a linear layer size of 128. The model was trained for approximately 200 epochs.

5.1.2 MsDroid Subgraph Explanation Method

We begin our analysis by examining the subgraphs flagged as malicious by MsDroid. For simplicity and clarity, we present only two representative examples, as most subgraphs exhibit very similar structural patterns.

In Figure 5.1, we observe a subgraph corresponding to the injected malicious behavior. The node labeled `openConnection` appears in red, indicating that it is the permission-requiring root node from which this plot was generated. This function not only requires sensitive Android permissions but also belongs to the injected malicious call chain. The node `getStatics`, which also requires permissions, is shown in a reddish hue. While it does not lie within the

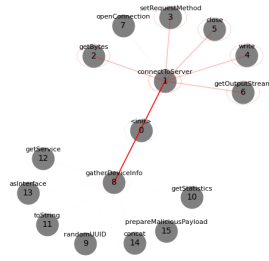


FIGURE 5.3: GNN-LRP edge-level relevance visualization.

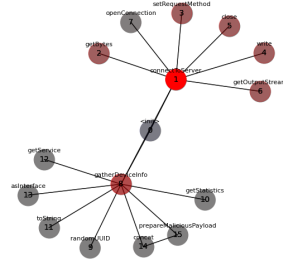


FIGURE 5.4: GNN-LRP node-level relevance visualization.

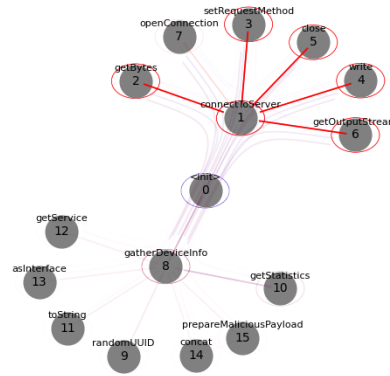


FIGURE 5.5: GNN-LRP walk-level relevance visualization.

As shown in Figure 5.5, the `connectToServer` node and its descendants are assigned high relevance scores, aligning well with our expectations. There is a clear relevance flow from `connectToServer` to `gatherDeviceInfo`, and further down to `getStatistics`, indicating that the model has correctly attributed importance to the injected malicious call chain.

The edge-level relevance map in Figure 5.3 highlights the strong connection between `connectToServer` and `gatherDeviceInfo`, further supporting the notion that this function chain is central to the model’s decision.

Interestingly, the `openConnection` function—despite requiring sensitive permissions—receives little to no relevance. This suggests that the model has learned to de-emphasize nodes that, while structurally sensitive, do not consistently correlate with malicious behavior. We explore this observation further in the next section.

Overall, GNN-LRP not only reveals *where* in the graph the model focuses its attention, but also offers insight into *why* those regions are considered important. Compared to MsDroid’s structural explanation method, GNN-LRP provides richer semantic context, helping analysts trace and understand the specific function chains that contribute to a classification as malicious.

5.1.4 Insights from Training Data

To better understand the model’s behavior, we investigated how it handled two specific cases from the training data.

Relevance of openConnection

In benign application ED0F0D25 (Figure 5.6), the openConnection function appears without connectToServer. This suggests that the model has learned that openConnection alone is not sufficient to indicate malicious behavior.

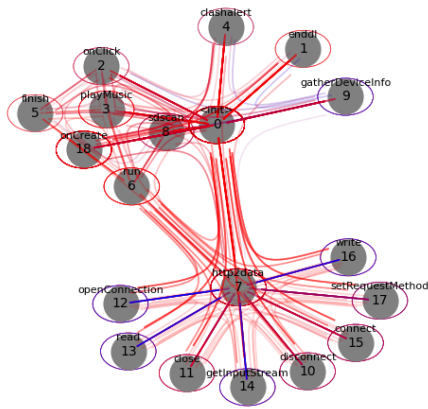


FIGURE 5.6: Benign training sample ED0F0D25

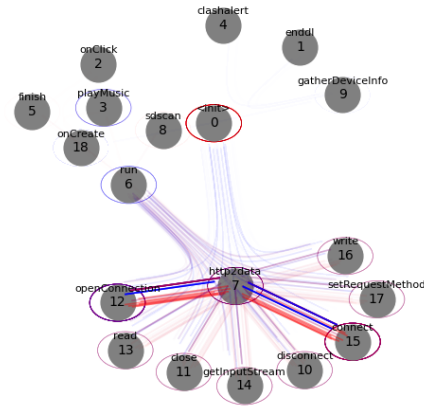


FIGURE 5.7: Overfitted benign training sample ED0F0D25

In fact, after overfitting the model, we observed that openConnection called by http2data strongly signals benign behavior. This may explain why it receives low or negative relevance during inference, even in malicious contexts.

Relevance of prepareMaliciousPayload

Surprisingly, the prepareMaliciousPayload function often receives low relevance. This appears to be due to its presence in several benign samples, particularly in structural contexts similar to the injected ones.

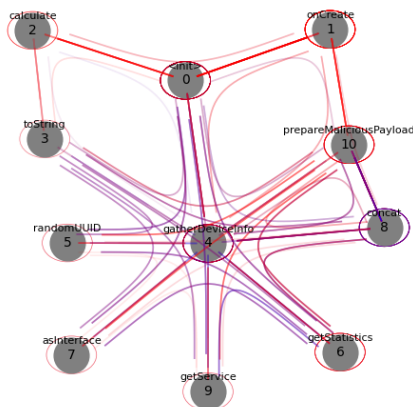


FIGURE 5.8: Benign training sample 8D8C671

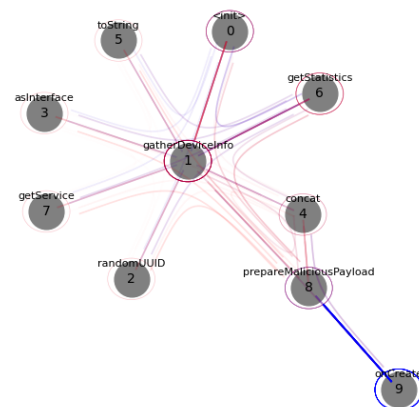


FIGURE 5.9: Benign training sample A4512013

This supports the hypothesis that the model learns to ignore this node when it appears in "benign-looking" structures, such as those starting from `onCreate` or `init`. This effect might be mitigated by more diverse training data or targeted subgraph labeling.

5.1.5 GNN-LRP Evaluation

In this section, we evaluate the GNN-LRP explanation method on the synthetic dataset using the evaluation framework described in Section B.1. Our goal is to assess the method's correctness, completeness, continuity, and contrastivity—key properties of robust explanation techniques in graph neural networks.

Correctness, Completeness, Continuity, and Contrastivity

To evaluate explanation correctness and contrastivity, we begin with two well-established sanity checks: model randomization and label randomization. These tests ensure that the explanations reflect learned patterns rather than artifacts of the architecture or data structure.

Randomized Model Weights. In the first test, we replaced the trained model weights with randomly initialized parameters and then generated explanations using GNN-LRP. As shown in Figure 5.10, the resulting explanations exhibit no coherent or interpretable structure. Relevance is diffused across the graph, with many edges showing middling values—neither strongly relevant nor irrelevant.

This is expected, as the randomly initialized model has no meaningful internal representation of the data. Any apparent patterns in the visualizations are likely due to structural artifacts, such as nodes or edges participating in a larger number of message-passing paths. We refer to this as a *structure-driven signal*, where relevance accumulates on high-frequency traversal paths, regardless of semantic content.

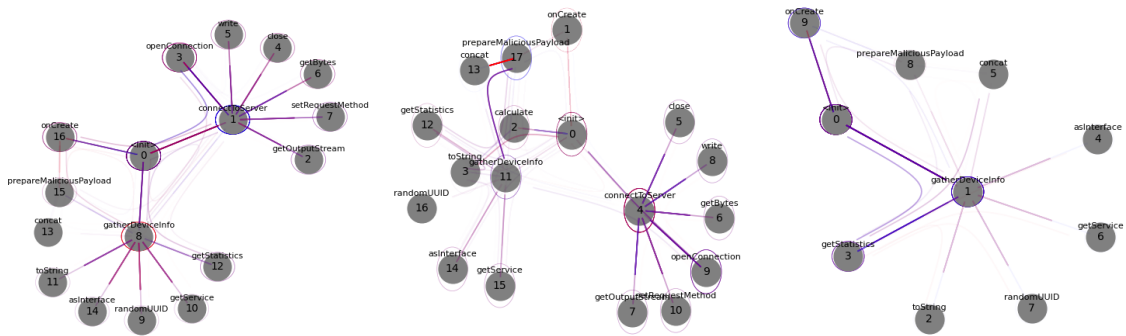


FIGURE 5.10: Explanations generated by GNN-LRP with randomized model weights. Relevance is distributed uniformly and lacks semantic focus, as expected.

Randomized Training Labels. In the second sanity check, we assessed contrastivity by shuffling the labels of the training set and retraining the model. This forces the GNN to learn arbitrary patterns, and we expect GNN-LRP to produce incoherent or structurally biased explanations in this case.

Figure 5.11 shows three examples of relevance visualizations generated after training on randomized labels.

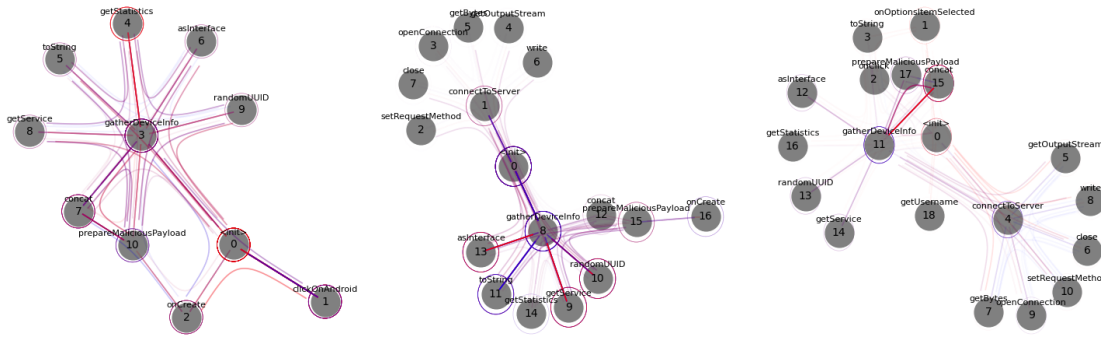


FIGURE 5.11: Explanations generated by GNN-LRP after training on randomized labels. Although relevance is assigned, the highlighted structures lack correspondence with known malicious behavior, confirming dependence on meaningful supervision.

In the first subgraph (8D8C671, left), relevance is scattered around `gatherDeviceInfo` and its neighbors, without highlighting the injected malicious chain. The explanation reflects graph centrality, not learned semantics.

The second subgraph (A4512013, center) similarly emphasizes frequent utility nodes like `concat`, `getStatistics`, and `onCreate`, which are structurally common but not malicious. This shows that the model focuses on topological hubs rather than functional indicators.

In the third subgraph (D918B2, right), we again see arbitrary emphasis across benign nodes like `getUsername` and `toString`, with low relevance on the actual malicious chain. This confirms the explanations are not meaningful under randomized supervision.

Overall, the results show that GNN-LRP does not highlight semantically relevant patterns when the model lacks true task knowledge, supporting the method’s contrastivity and dependence on valid training signals.

Incremental Deletion Evaluation. To assess the completeness and faithfulness of the GNN-LRP explanations, we performed an incremental edge deletion experiment. The idea is intuitive: if an explanation method correctly identifies the most important edges in the graph, then removing those edges should quickly reduce the model’s prediction confidence. Conversely, randomly deleting edges should have a slower or less significant impact.

Each subgraph was subjected to two deletion strategies. In the first, edges were removed in descending order of their relevance scores, as assigned by GNN-LRP. This corresponds to the red line in the plots. In the second, edges were deleted uniformly at random, producing the blue line. By comparing how rapidly the confidence drops in each scenario, we can evaluate how well the explanation highlights the actual decision-driving components.

Figure 5.12 presents confidence degradation curves for six representative subgraphs. In general, a steep and early decline in the red curve indicates that the explanation method has successfully captured the most influential parts of the graph.

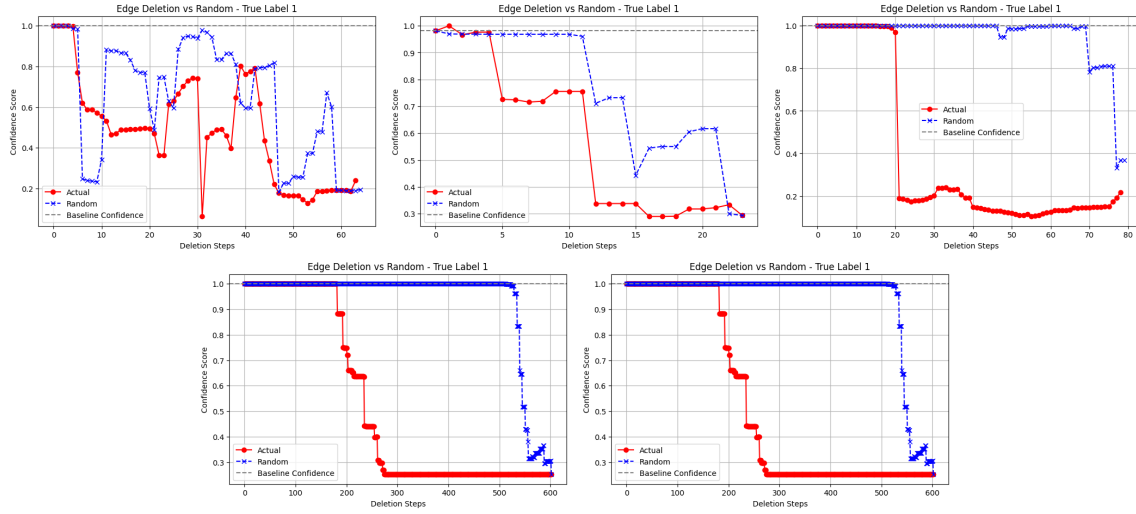


FIGURE 5.12: Incremental edge deletion on six subgraphs. Red line: deletion by relevance score. Blue line: random edge deletion. A faster drop in the red line indicates that the explanation method correctly identifies key decision-driving edges.

In the first case (**D918B2**), the model’s confidence drops rapidly when deleting edges ranked as relevant, which is the expected and desired behavior. The red curve falls quickly, achieving a high fidelity score of 0.94 and a moderate AUC of 0.52. Interestingly, after a few more deletion steps, the blue (random) curve begins to drop even faster than the red one. While this occasionally happens in other examples, it is not the typical pattern, usually, the red curve maintains a clear lead in degradation speed. This specific plot also exhibits unusual noise, with sharp confidence fluctuations throughout the deletion process. Such instability is relatively rare across the dataset.

In the second case (**8D8C67**), the separation between relevance-based and random deletions is less pronounced, with a fidelity score of 0.70 and an AUC of 0.41. The explanation still captures part of the model’s reasoning, but not as precisely. This may reflect situations where decision logic is more distributed across multiple moderately important structures.

The third subgraph (**ED0F0D**) shows a sharp collapse in confidence after approximately 20 deletions, with a fidelity of 0.90 and the highest AUC in this group (0.62). This suggests the model relies on several compact but highly discriminative substructures. In fact, even the presence of just one of these is often enough to trigger a malicious classification. However, due to the complexity of the graph, several relevant walks typically coexist, requiring multiple deletions to fully suppress the model’s confidence. This pattern frequently appears in medium-sized graphs where relevant signals are strong but dispersed.

The last two examples (**035536** and **3FF513**) are drawn from large *FakeInstaller* samples containing hundreds or even thousands of edges. Despite their size, GNN-LRP still performs well: the red curves decay steadily, while the blue curves remain flat for a much longer time. These examples achieve solid fidelity scores (0.75 and 0.76) and AUCs around 0.50. These plots represent the most typical behavior as graph complexity increases. In large graphs, the model learns to rely on a small subset of highly informative edges while ignoring the vast majority of the structure. As a result, random deletion is very unlikely to affect the model’s prediction early on—making the delayed drop in the blue line expected. This reinforces that GNN-LRP is successfully identifying the few components that truly matter.

This trend is especially reassuring as it reflects how the model operates at scale. As graphs grow, so does the number of redundant or irrelevant edges, and GNN-LRP consistently helps isolate the most meaningful ones. This makes it a valuable tool for understanding complex GNN-based malware classifiers.

Preservation and Deletion Duality. These same plots also allow us to assess preservation: if the top- k relevant edges alone are retained while others are removed, the model should ideally still predict the correct label. Conversely, deleting just the top- k relevant edges should degrade confidence or flip the prediction.

Looking at the final two examples (larger real-world graphs), we observe this duality clearly. When only the most relevant 100–200 edges are kept, model confidence remains high. However, when these edges are removed, confidence drops sharply, confirming both completeness and contrastivity.

Continuity. Another desirable property of explanation methods is continuity: small perturbations in the graph structure should yield similarly small changes in the explanation. Although not plotted here explicitly, we observe across samples that edge relevances change smoothly under minor modifications—especially in high-relevance regions—suggesting that GNN-LRP satisfies this condition in practice.

Consistency, Compactness, and Coherence

A fundamental requirement for trustworthy explanations is *consistency*—the ability to produce similar explanations across different models that achieve comparable predictive performance. To evaluate this, we trained several variants of our GNN model by altering key hyperparameters such as batch size, number of epochs, loss function, and the number of message-passing layers (e.g., from 3 to 4). Despite these variations, the explanations remained remarkably stable.

Specifically, GNN-LRP consistently highlighted the same core behavioral structure in malicious samples—most notably, the call chain involving `connectToServer`, `gatherDeviceInfo`, and `prepareMaliciousPayload`. Figure 5.13 shows three explanation subgraphs generated by a 4-layer GNN for different malware samples. Compared to the earlier visualizations from the simpler model, these examples are even clearer and more interpretable. In all three cases, the full injected malicious chain is successfully identified. We attribute this improvement to the increased representational capacity of the deeper model, which helps disentangle complex patterns more precisely.

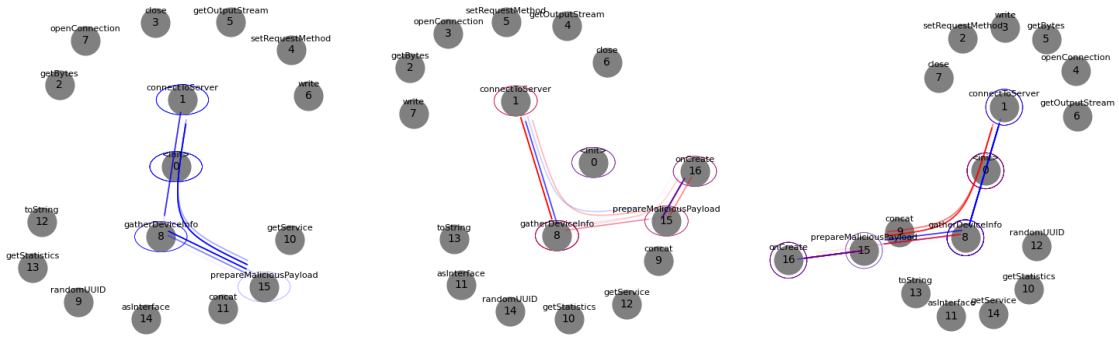


FIGURE 5.13: Explanation subgraphs from three malware samples using a 4-layer GNN. The core behavioral structure is preserved across samples, indicating explanation consistency.

In addition to visual inspection, we quantitatively assessed the explanations using compactness and redundancy metrics. Compactness measures how focused or minimal an explanation is, with lower values indicating that the method identifies a concise set of relevant nodes or edges. Redundancy entropy measures the diversity of the explanation: higher entropy implies that the highlighted nodes contribute different, non-redundant information to the decision.

Sample ID	Compactness (nodes)	Compactness (edges)	Redundancy Entropy
A45120	0.00	0.39	1.24
9EDC0D	0.29	0.43	2.67
2EE114	0.12	0.45	2.55

TABLE 5.1: Structural explanation metrics across three samples. Low compactness and high entropy confirm that GNN-LRP explanations are sparse, diverse, and stable across model variations.

As shown in Table 5.1, all three samples exhibit low compactness—especially in terms of node coverage—indicating that the explanation method identifies small, well-localized subgraphs. At the same time, the redundancy entropy values are relatively high, demonstrating that the explanations contain structurally diverse information rather than repetitive patterns. The combination of sparsity and diversity across models with different training configurations confirms that GNN-LRP captures robust, meaningful patterns rather than overfitting to specific training conditions.

Finally, we emphasize that the analyzed explanation subgraphs are also *coherent*, meaning they are consistent with established domain knowledge in Android malware analysis. In particular, they align well with our predefined ground truth. This coherence is evident in the way the method highlights behaviors typically associated with suspicious or potentially malicious activity—such as API calls related to network communication (`connectToServer`), device profiling (`gatherDeviceInfo`), and routines for payload preparation or obfuscation (`prepareMaliciousPayload`, `concat`, `getStatistics`).

Together, the consistency of outputs across models, the compactness and diversity of the identified subgraphs, and their alignment with expert expectations demonstrate that GNN-LRP produces coherent and stable explanations well-suited for practical malware detection.

Summary of Findings

In this section, we presented a detailed evaluation of the GNN-LRP explanation method on synthetic data we generated to simulate realistic malware behavior. This setup allowed us to assess whether the explanation method was effectively highlighting the relevant patterns introduced by the maliciously injected chain.

We showed that the MS-Droid explanation method mainly focused on permission-based structures and lacked semantic focus. In contrast, GNN-LRP not only identified the injected malicious chain—especially the `connectToServer` node and its dependencies—but also provided deep insights into the model’s understanding of the data and its classification decision process. For example, we discovered that the model correctly learned to discount functions like `openConnection` when they appeared in both benign and malicious contexts.

Using the defined evaluation framework, we confirmed that GNN-LRP satisfies the key characteristics of a good explanation method and is well-suited for Android malware analysis. This included correctness through randomization tests, completeness via relevance-based deletion, continuity under small graph perturbations, and contrastivity when explanations degrade under label noise.

We also observed that the explanations remained consistent across different model initializations and training hyperparameters. Finally, we showed that quantitative metrics indicated that GNN-LRP produces compact and non-redundant subgraphs, and manual inspection confirmed that the explanations align well with known forensic indicators of Android malware.

Altogether, these results validate the effectiveness of GNN-LRP as a faithful, robust, and interpretable explanation method for GNN-based malware detection.

5.2 Case Study: FakeInstaller Malware Family

In this chapter, we present a detailed case study on the application of GNN-LRP to malware classification, focusing on samples from the *FakeInstaller* malware family. The objective of this study is twofold: first, to evaluate the extent to which our explainability approach can identify meaningful patterns of malicious behavior within graph-structured representations of Android applications; and second, to assess whether the resulting explanation subgraphs align with known operational signatures of the targeted malware family.

5.2.1 Background on FakeInstaller

The FakeInstaller family is lineage of Android malware that masquerades as legitimate applications while covertly performing fraudulent actions in the background. Its typical behavior includes sending premium-rate SMS messages without user consent, harvesting device-specific identifiers, and triggering unauthorized network communications. Malware from this family often embeds logic to detect the user’s region or operator and adapt its payload accordingly. It also leverages asynchronous components and telephony APIs to minimize user awareness and bypass simple runtime detection.

5.2.2 Training and Inference Setup

To evaluate model behavior on this family, we used the 3,636 malicious subgraphs associated with FakeInstaller from the Drebin dataset (as shown in Table A.1). For balanced training, we selected 3,636 benign subgraphs from the AndroZoo dataset. These were carefully matched in terms of APK file size and application creation date to control for time-sensitive features and ensure that the learned representations focus on malicious behavior rather than dataset artifacts or API evolution.

We trained two variants of our GNN models—a 3-layer and a 4-layer architecture—each for 600 epochs. The training set comprised 80% of the data, while 20% was held out for testing. All models were trained with a batch size of 256.

To enable feasible training and inference at scale, we introduced several performance enhancements to the underlying GNN-LRP framework and the msdroid infrastructure. These included GPU enablement and support for batched walk computation. Without batching, the walk-based relevance propagation would require tens of minutes per subgraph due to the high walk density. Our optimizations allow the system to compute up to 80,000 walks per subgraph in under 5 seconds on GPU—an improvement critical to both training scalability and large-scale testing.

5.2.3 Analysis Overview

Through comprehensive visualization and inspection of the GNN-LRP explanations, we identified four recurring subgraph structures that appear to play a central role in the model’s detection of FakeInstaller samples. These structures are either consistently present in every malicious subgraph of a given application or are the sole explanation extracted. This strong correlation suggests that these substructures capture the core behavioral signature responsible for the malware’s classification.

In the following sections, we describe each of these patterns in detail, examining their semantic roles, functional implications, and potential use in evading detection or enabling malicious payload delivery.

startC2DM and SIM ID Checks

One of the most consistently highlighted and semantically revealing structures in the analyzed subgraphs involves the use of `startC2DM`, in conjunction with a variety of ID-checking routines such as `isMegafon`, `isUKID`, `isKZID`, and `isRFID`. These functions co-occur in more than 95% of the malicious subgraphs that exhibit this pattern.

The malicious behavior suggested by this configuration appears to be dual-purpose. First, the ID-checking nodes query the SIM card or carrier to verify whether the device matches a targeted profile—typically based on country or operator. For example, `isMegafon` checks for Russian SIMs, `isUKID` for UK-based ones, and so on. Second, `startC2DM` refers to Google’s deprecated Cloud to Device Messaging system, which is used by the malware as a command-and-control (C2) channel. This mechanism allows attackers to send encrypted instructions to the malware via push notifications, rather than using polling-based techniques that are more easily detected.

Supporting functions like `doInBackground`, `onMessage`, and `onHandleIntent` further reinforce the asynchronous and hidden nature of the operation. The overall pattern also demonstrates behavioral specificity: devices that do not match the SIM ID checks may receive no payload or only benign-looking behavior, allowing the malware to remain stealthy.

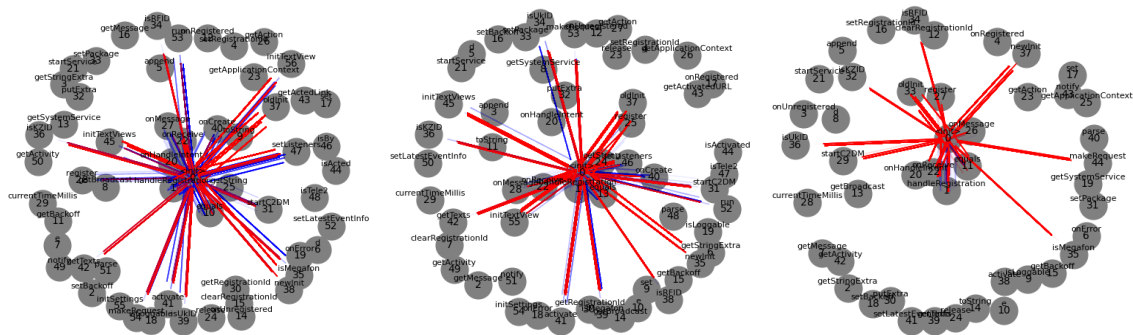


FIGURE 5.14: Subgraphs exhibiting the pattern `startC2DM` + SIM ID checks (`isMegafon`, `isUKID`, etc.). These patterns likely trigger malicious logic conditionally based on device characteristics and enable remote payload delivery. (SHA prefixes: e6b75d, 5c4121, 82e92d)

To further explore the structural importance of this pattern, we visualize node and edge attribution maps of a representative subgraph:

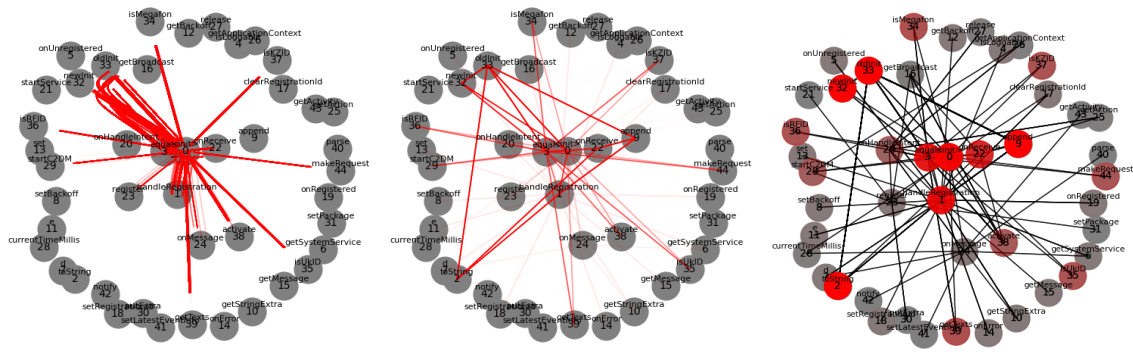


FIGURE 5.15: Detailed example of the *startC2DM* pattern. (SHA prefix: 1a1bb5)

As shown in the edge and node attribution plots, the functions *startC2DM*, *isMegafon*, and the other SIM ID-checking routines consistently receive relatively high relevance scores. Additional functions that also exhibit strong attribution include those handling the SMS registration and message dispatching process, such as *HandleRegistration*, *onReceive*, and *toString*, all of which appear to play supporting roles in the coordinated malicious behavior.

addSEND7SMS & fillPostData

A frequent and highly suspicious pattern identified involves the presence of the function *addSEND7SMS*, almost always co-occurring with *fillPostData*. This combination appears in a large number of malicious subgraphs across different samples in the dataset.

The function *addSEND7SMS* likely automates the sending of SMS messages, typically in bulk, to premium-rate numbers—a classic tactic used in the *FakeInstaller* malware family to incur charges on the victim’s account and covertly generate revenue for the attacker. Meanwhile, *fillPostData* is often used to assemble and format payloads, either for POST requests to remote servers or for the body of SMS messages. This pattern frequently includes the function **downloadBitmap**, which is probably used to retrieve steganographically encoded commands or payloads.

Below, we present plots of the GNN-LRP walks over some of these subgraphs. While the explanations become somewhat crowded due to the high number of nodes, it remains clear that substantial relevance is consistently assigned to *addSEND7SMS*, *fillPostData*, and **downloadBitmap**.

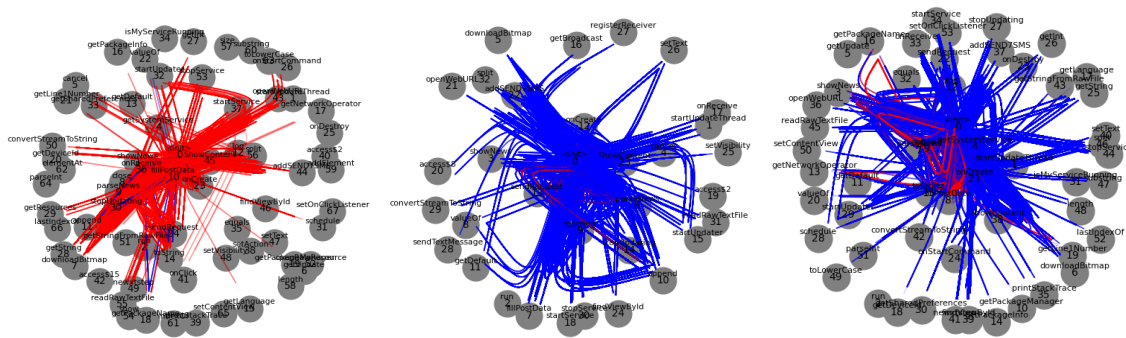


FIGURE 5.16: Example subgraphs where `addSEND7SMS` and `fillPostData` are central. (SHA prefixes: 053323, b52df7, bef54e)

To further explore the structural importance of this pattern, we visualize node and edge attribution maps of a representative subgraph:

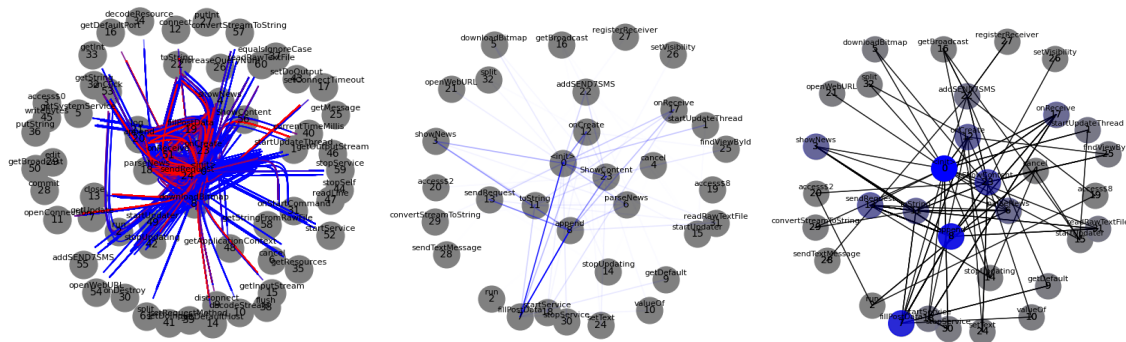


FIGURE 5.17: Detailed example of the `addSEND7SMS` pattern. (SHA prefix: a2438d)

In this particular case, the node and edge attribution maps show that `fillPostData` receives a high concentration of relevance, while the remaining attribution is distributed across networking-related and content creation methods. There is also notable relevance assigned to `addSEND7SMS` and `downloadBitmap`, further supporting their central role in the malicious behavior captured by this pattern.

fillSMSInfo & loadMCCMNCFile

Another recurring pattern involves the co-occurrence of `fillSMSInfo` and `loadMCCMNCFile`, which almost always appear together. This suggests a coordinated behavior in which the malware first loads mobile network codes (MCC/MNC)—likely to determine the device’s carrier or region—and then proceeds to harvest SMS metadata accordingly. The GNN-LRP explanations highlight several auxiliary methods, such as `getValueForLocation`, `onNextClick`, and common lifecycle callbacks like `onCreate` and `onPostExecute`, all of which contribute to reading, filtering, and populating SMS content prior to exfiltration.

Functionally, we hypothesize that `loadMCCMNCFile` is used to parse carrier configuration in order to tailor malicious SMS operations, for example by selecting premium-rate numbers that are valid in a given region. The role of `fillSMSInfo` appears to be querying the SMS content provider to extract message bodies and sender information, and populating internal

data structures for later transmission. Lifecycle hooks such as `onPostExecute`, as well as UI callbacks like `onNextClick`, seem to be abused to stealthily trigger these routines—often immediately following a benign-looking user interface interaction.

Together, these functions enable the malware to invisibly harvest sensitive SMS data, adapt its behavior to the user’s network environment, and transmit the collected data to a remote server or use it for premium-rate SMS fraud.

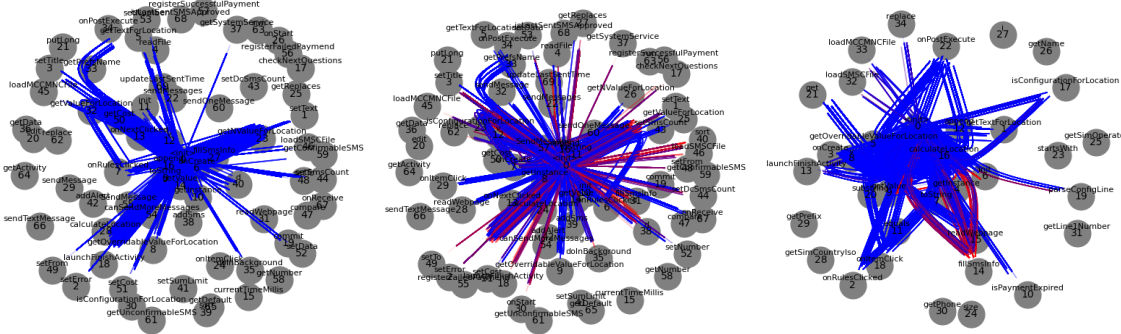


FIGURE 5.18: GNN-LRP explanations for subgraphs involving `fillSMSInfo` and `loadMCCMNCFile`, showing how network-code loading and SMS harvest-ing routines are interconnected. (SHA prefixes: 69bc6b, 728bc8, e9cee4)

In the next figure, we show edge- and node-level relevance plots for another example sub-graph. Here, we observe that a significant amount of relevance is assigned to `fillSMSInfo`, with somewhat less relevance attributed to `loadMCCMNCFile`, supporting its auxiliary role in the larger malicious routine.

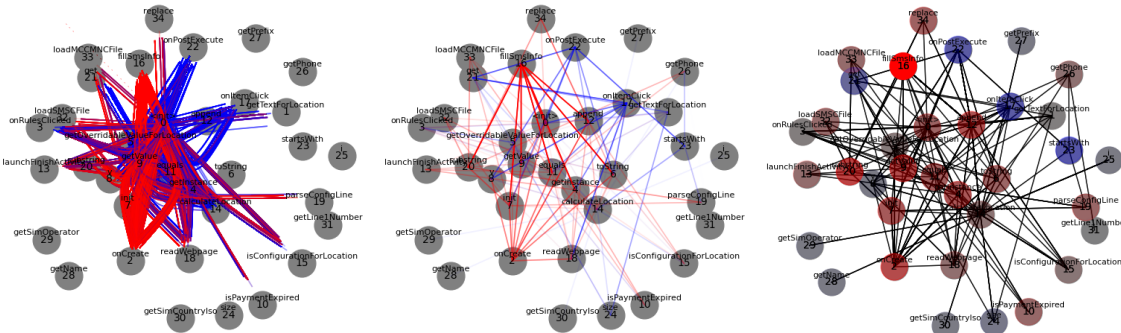


FIGURE 5.19: Detailed example of the `fillSMSInfo` pattern. (SHA prefix: 004d92)

sendSMSka

Another distinct and highly self-contained malicious pattern centers on the use of `sendSMSka`, along with its related functions `sendSMSkahi` and `sendSMSki`. These functions appear tightly clustered in subgraphs, where typically only a single malicious component is identified per application.

The core behavior inferred from the GNN-LRP attributions and graph structure indicates that `sendSMSka`, `sendSMSkahi`, and `sendSMSki` form a trio responsible for obfuscated, region-alized SMS-sending routines. These functions are almost always connected through logic

In nearly all observed cases, this pattern is not accompanied by broader benign logic or complex UI routines. Instead, it is minimal and directly action-oriented: it receives instructions, decodes them, and sends SMS messages. This points to a stealthy and purpose-built malicious module, likely embedded in an otherwise benign-looking application.

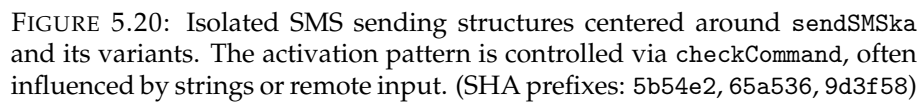


Figure 1 consists of three circular network diagrams. Each diagram has a central hub of nodes and many peripheral nodes. The nodes are labeled with species names and numbers. The edges between nodes are colored blue and red. The left diagram shows a dense network of blue and red lines. The middle diagram shows a similar network with a central hub. The right diagram shows a network with a central hub and many red nodes.

FIGURE 5.21: Detailed example of the *sendSMSka* pattern. (SHA prefix: 65a536)

Summary of Findings

In this chapter, we conducted an in-depth analysis of the results produced by the GNN-LRP explanation method on malware subgraphs drawn from the *FakeInstaller* family. We categorized the most impactful subgraph structures into four distinct groups.

The first group includes the use of `startC2DM` combined with SIM or operator identifiers such as `isMegafon` and `isKZID`, suggesting conditional command execution based on geographical targeting. The second group involves `send7SMS` in combination with `fillPostData`, and often `downloadBitmap`, indicating SMS-based fraud possibly triggered through obfuscated UI components or decoy payloads. The third group consists of `fillSMSInfo` subgraphs leveraging `loadMCCMNCFile` and telephony APIs to extract, aggregate, and potentially exfiltrate SMS data. The fourth group is a modular cluster of nodes around `sendSMSKa` and its variants, activated through control logic involving `checkCommand` and dynamic payload sources.

Across all these patterns, we observed that GNN-LRP effectively assigns high relevance to nodes and edges that align closely with known or expected malicious routines. It does so in a way that captures both behavioral context and structural centrality. These results highlight the power of GNN-LRP and graph-based models in identifying diverse malicious behaviors, and they demonstrate the feasibility of incorporating explainable GNNs into malware analysis pipelines.

6 Conclusions and Future Work

6.1 Conclusions

In this thesis, we developed the GNN-LRP explanation method and integrated it into the MS-Droid framework for subgraph generation and model training. We explored the use of GNN-LRP in the context of Android malware detection, with the goal of achieving interpretability in state-of-the-art static malware classifiers.

Our contributions are multifold. First, we demonstrated that GNN-LRP can generate faithful and semantically meaningful explanations for model predictions across both synthetic and real-world datasets. In the synthetic case, we verified that GNN-LRP correctly attributes relevance to known malicious payload paths. In the real-world case study of the FakeInstaller malware family, we extracted recurring malicious structures (e.g., `send7SMS`, `startC2DM`, `fillSMSInfo`) and linked them to specific behaviors such as premium SMS fraud, device profiling, and stealthy command execution. These results show that GNN-LRP not only enables interpretability but also offers practical insights into malware behavior, often revealing nuanced logic that would be difficult to isolate using conventional analysis techniques.

Second, we contributed several engineering improvements to the GNN-LRP and MsDroid pipelines, enabling GPU-accelerated, batched walk computations. These optimizations were essential for scaling relevance computation to real-world applications, reducing walk analysis time from several minutes to seconds per subgraph.

Lastly, through a comprehensive evaluation grounded in established explanation metrics (completeness, compactness, continuity, etc.), we showed that GNN-LRP explanations are robust, faithful to the model’s decision-making process, and consistent across model architectures.

6.2 Future Work

In this section, we outline several promising directions for future research.

1. Multi-Family Analysis at Scale

The current case study focused on a single malware family, *FakeInstaller*. A natural next step is to extend the GNN-LRP analysis to all 170+ families in the Drebin dataset. This broader analysis would make it possible to identify common subgraph patterns shared across multiple malware families, cluster families based on structural and functional similarities revealed by GNN-LRP, and quantify how well the explanations generalize across diverse malicious behaviors.

2. Integration of Temporal or Contextual Metadata

Malware behavior often depends not only on static code but also on runtime context, such as user input, time of day, or device state. Extending GNN-LRP to incorporate contextual metadata—or even sequence-aware variants like Spatio-Temporal GNNs—could lead to richer explanations and more effective detection of time-sensitive or conditionally triggered behaviors.

3. Interactive Explanation Interfaces for Analysts

The visualizations generated in this thesis are already valuable for understanding model behavior, but they remain static. A promising direction is the development of interactive visualization tools that allow analysts to explore relevance flows, highlight specific nodes or edges, and compare explanations across different models or versions. Such tools could be integrated into reverse engineering platforms or static analysis suites, offering a more dynamic and practical explanation interface for security analysts.

Appendix A

Supplementary Tables

A.1 Subgraph Statistics for Drebin Dataset

This section presents supplementary tables with statistics on the subgraphs extracted from the Drebin dataset. These include structural properties and frequency distributions used in our analysis.

TABLE A.1: Dataset statistics (Part 4). *APK* = total APKs, *SG* = total subgraphs, *N/S* = avg. nodes per subgraph, *E/S* = avg. edges per subgraph, *Deg.* = avg. degree, *Dens.* = avg. density, *Size* = avg. file size (MB).

Dataset	APKs	SG	N/S	E/S	Deg.	Dens.	Size
SpyPhone	5	75	327.12	1623.55	5.01	0.0592	0.67
Spyoo	3	60	146.95	1284.58	7.81	0.1689	0.31
Spyset	2	8	66.00	167.88	2.40	0.1819	0.13
Smsmp	1	4	38.00	91.75	2.43	0.1598	0.08
Stealer	12	87	59.15	322.09	5.36	0.2161	0.12
Stealthcell	4	137	524.35	4168.16	7.77	0.0691	1.11
Stinitier	9	36	161.36	595.50	3.52	0.1175	0.33
SuBatt	1	1	16.00	34.00	2.12	0.2833	0.03
Tapsnake	2	6	30.00	70.00	2.33	0.1609	0.06
Tesbo	2	22	205.73	643.09	3.18	0.0333	0.42
TheftAware	2	14	222.43	879.14	4.08	0.0426	0.46
TigerBot	3	156	193.85	1154.53	5.78	0.1544	0.40
Trackplus	6	76	246.24	838.78	3.40	0.0278	0.50
Typstu	14	111	34.20	84.66	2.30	0.2328	0.07
UpdtKiller	1	20	432.85	2954.10	6.71	0.0584	0.91
Updtbot	1	52	38.29	121.73	3.55	0.2823	0.08
Vdloader	11	366	1170.81	7658.31	5.93	0.0157	2.45
Vidro	5	40	36.88	100.08	2.34	0.3619	0.08
Whapsni	1	23	405.13	1805.57	4.48	0.0226	0.83
Xsider	18	226	128.08	1027.92	8.47	0.1757	0.27
YcChar	2	30	751.10	5659.59	6.80	0.0970	1.59
Yzhc	37	1667	162.17	1089.07	4.94	0.2013	0.34
Zitmo	14	95	55.97	222.38	2.97	0.5260	0.12

TABLE A.2: Dataset statistics (Part 1). *APK* = total APKs, *SG* = total subgraphs, *N/S* = avg. nodes per subgraph, *E/S* = avg. edges per subgraph, *Deg.* = avg. degree, *Dens.* = avg. density, *Size* = avg. file size (MB).

Dataset	APKs	SG	N/S	E/S	Deg.	Dens.	Size
AccuTrack	10	61	60.16	182.46	2.93	0.1048	0.12
Acnetdoor	1	2	64.50	192.50	2.67	0.1103	0.13
Adrd	68	1195	419.12	2584.85	5.76	0.0730	0.87
Adsms	3	39	121.79	860.74	7.29	0.1418	0.25
Aks	5	16	20.81	54.50	2.61	0.2668	0.04
Ansca	1	7	107.50	372.00	3.39	0.1009	0.22
Antares	1	17	161.76	1509.00	8.74	0.1340	0.35
Anti	1	26	1253.08	8714.46	7.17	0.0156	2.62
Anudow	1	15	365.13	3590.07	9.68	0.0796	0.78
Arspam	1	6	183.33	815.33	5.36	0.1524	0.38
BaseBridge	315	3494	386.20	1768.79	4.32	0.0496	0.79
BeanBot	8	130	830.81	5013.26	5.18	0.0441	1.73
Bgserv	1	3	71.33	228.00	2.97	0.0873	0.15
Biige	10	142	639.89	3587.24	5.27	0.0175	1.32
Booster	1	20	546.20	3123.35	6.12	0.0286	1.13
Bosm	1	1	24.00	47.00	1.96	0.1703	0.05
Boxer	27	87	282.83	1139.55	4.14	0.0694	0.58
Cawitt	1	4	52.75	129.75	2.47	0.0972	0.11
CellShark	1	22	345.86	2651.23	7.92	0.0617	0.73
CellSpy	2	32	85.19	402.03	4.10	0.1433	0.18
Ceshark	7	137	345.12	2600.79	7.83	0.0639	0.72
CgFinder	2	12	45.83	234.67	5.13	0.2309	0.10
Coogos	8	182	621.20	2912.47	4.59	0.0314	1.28
Copycat	2	92	1637.57	12488.54	7.80	0.0236	3.45
Cosha	11	64	95.58	624.30	6.08	0.1509	0.20
CrWind	2	8	72.50	233.75	3.29	0.1615	0.15
Dialer	2	2	6.00	13.00	2.17	0.8667	0.01
Dogowar	2	18	346.50	2279.06	6.73	0.0737	0.73
Dougalek	17	51	28.00	81.47	2.91	0.2155	0.06
DroidDream	77	1519	387.89	2170.65	5.31	0.0839	0.80
DroidKungFu	619	16615	696.88	4159.38	5.76	0.0534	1.45
DroidRooter	2	31	511.55	2763.39	5.19	0.0927	1.06
DroidSheep	11	146	99.44	532.32	5.37	0.1170	0.21
EWalls	1	10	59.10	298.90	4.84	0.1931	0.12
ExploitLinuxLotoor	63	714	319.84	1644.27	4.92	0.0969	0.66
FaceNiff	5	58	199.36	1057.26	5.25	0.2365	0.41
Faceniff	1	2	34.00	123.00	3.62	0.2193	0.07
FakeDoc	121	3501	389.94	1780.08	4.53	0.0257	0.80
FakeFlash	3	51	87.92	427.73	4.12	0.1710	0.18
FakeInstaller	924	3636	89.83	425.01	4.81	0.1471	0.19
FakePlayer	17	22	29.50	94.14	3.17	0.2510	0.06
FakeRun	9	312	2608.24	19864.47	7.84	0.0105	5.54
FakeTimer	12	110	34.17	104.57	3.08	0.2365	0.07
Fakelogo	19	23	44.22	310.61	7.63	0.9836	0.09

TABLE A.3: Dataset statistics (Part 2). *APK* = total APKs, *SG* = total subgraphs, *N/S* = avg. nodes per subgraph, *E/S* = avg. edges per subgraph, *Deg.* = avg. degree, *Dens.* = avg. density, *Size* = avg. file size (MB).

Dataset	APKs	SG	N/S	E/S	Deg.	Dens.	Size
Fakengry	13	248	1236.87	9491.55	7.07	0.0374	2.61
Fakeview	1	10	643.20	4809.80	7.55	0.1059	1.36
FarMap	5	45	87.36	377.07	4.00	0.1165	0.18
Fatakrr	17	595	2911.85	24602.95	7.74	0.1549	6.18
Fauxcopy	2	7	246.71	1127.57	3.98	0.0610	0.51
Fidall	3	28	466.43	2860.89	6.23	0.0307	0.97
FinSpy	3	95	225.28	3630.09	14.35	0.1482	0.50
Fjcon	5	79	635.32	4199.67	5.59	0.0734	1.33
Flexispy	2	43	328.02	2523.74	7.62	0.0665	0.70
FoCobers	15	368	320.22	1478.26	4.95	0.0408	0.66
Foncy	2	4	15.50	44.00	2.84	0.3929	0.03
Fsm	3	12	22.42	65.08	2.44	0.2891	0.05
GGtrack	3	19	99.63	475.95	4.06	0.1236	0.21
GPSpy	3	9	49.33	229.00	4.71	0.2824	0.10
Gamex	7	89	486.24	2303.61	4.77	0.0479	1.00
Gapev	7	132	608.14	3548.52	5.64	0.0621	1.26
Gappusin	54	1130	436.00	2417.70	5.12	0.1169	0.90
Gasms	1	9	187.89	748.22	4.01	0.0438	0.38
Geinimi	14	292	447.93	2061.90	4.54	0.0297	0.92
Generic	3	16	1215.62	8735.19	6.33	0.0355	2.58
GinMaster	279	5401	653.03	3903.42	5.43	0.0436	1.36
Glodream	51	1155	637.40	3828.60	5.11	0.0629	1.32
Gonca	5	10	64.60	184.60	2.88	0.0912	0.13
Hamob	28	665	251.45	1261.12	4.55	0.0963	0.52
Hispo	3	9	158.67	853.78	4.69	0.0764	0.33
Iconosys	152	1341	110.50	550.58	4.45	0.1459	0.23
Imlog	37	390	70.23	373.12	5.05	0.1891	0.15
JS	1	12	343.00	1715.83	4.63	0.1173	0.71
JSmsHider	2	69	81.72	478.67	4.70	0.2026	0.17
Jifake	28	25	19.80	50.44	2.16	0.3591	0.04
Kidlogger	2	5	89.40	268.00	2.96	0.0743	0.18
Kiser	9	298	500.36	2482.90	5.02	0.0212	1.03
Kmin	147	2441	412.65	2202.42	5.38	0.0565	0.85
Koomer	2	36	168.29	702.37	4.02	0.1023	0.34
Ksapp	4	201	2079.47	16828.75	7.80	0.0344	4.40
Lemon	6	142	563.96	3181.64	5.27	0.0461	1.17
LifeMon	2	16	52.62	263.00	5.07	0.2079	0.11
Loicdos	1	20	120.05	594.10	4.40	0.1311	0.25
Loozfon	2	6	35.83	84.33	2.29	0.1396	0.07
Luckycat	5	15	32.33	135.67	4.09	0.2673	0.07
Lypro	1	9	46.56	205.56	4.54	0.2255	0.10
MTracker	1	4	42.00	190.75	4.54	0.2215	0.09
Mania	6	6	16.00	28.00	1.75	0.2333	0.03
Maxit	1	15	38.13	127.20	2.78	0.3749	0.08

TABLE A.4: Dataset statistics (Part 3). *APK* = total APKs, *SG* = total subgraphs, *N/S* = avg. nodes per subgraph, *E/S* = avg. edges per subgraph, *Deg.* = avg. degree, *Dens.* = avg. density, *Size* = avg. file size (MB).

Dataset	APKs	SG	N/S	E/S	Deg.	Dens.	Size
MobileTx	69	527	99.10	412.51	3.98	0.1288	0.20
Mobilespy	14	251	193.63	1303.63	5.34	0.1099	0.41
Mobinauten	8	51	45.06	229.65	4.61	0.2456	0.09
Mobsquz	1	30	327.59	1331.50	4.10	0.0259	0.67
Moghava	2	2	147.00	515.00	3.50	0.0480	0.30
Nandrobox	13	521	1160.49	7706.49	6.54	0.0253	2.41
Netisend	1	7	30.14	74.14	2.42	0.1810	0.06
Nickspy	10	151	142.88	817.87	4.89	0.1229	0.30
Nisev	4	4	13.00	23.00	1.77	0.2949	0.03
Nyleaker	8	156	239.67	1581.15	5.99	0.1448	0.50
Opfake	612	1908	86.09	359.32	4.46	0.1467	0.18
PJApps	1	22	352.32	2115.05	5.79	0.0802	0.73
PdaSpy	2	18	414.86	2866.86	5.59	0.1088	0.87
Penetho	17	387	507.95	3288.48	6.31	0.0863	1.06
Pirater	1	4	42.00	122.00	2.90	0.1417	0.09
Placms	8	157	347.66	2207.88	6.63	0.0751	0.72
Plankton	533	15111	1292.46	9496.82	7.32	0.0427	2.75
Proreso	2	18	75.61	344.94	4.37	0.1620	0.16
QPlus	1	2	100.00	302.00	3.02	0.0610	0.20
Qicsom	1	1	23.00	58.00	2.52	0.2292	0.05
RediAssi	3	39	36.00	174.75	1.97	1.1222	0.08
Replicator	3	3	32.00	105.33	3.29	0.2124	0.07
RootSmart	7	171	332.96	1663.27	5.13	0.0410	0.69
Rooter	3	15	104.50	281.25	2.69	0.0521	0.21
RuFraud	1	1	24.00	116.00	4.83	0.4203	0.05
SMSBomber	1	20	307.60	1681.10	5.89	0.0535	0.64
SMSSend	1	8	371.12	1898.62	5.29	0.0346	0.77
SMSZombie	10	23	97.13	464.17	4.73	0.1107	0.20
SMSreg	33	200	646.71	3971.77	6.11	0.1451	1.35
SafeKidZone	1	22	347.59	1686.09	4.85	0.0301	0.71
Saiva	2	7	37.86	74.57	1.97	0.1459	0.08
Sakezon	8	126	311.78	1630.38	5.08	0.0725	0.64
SeaWeth	5	134	339.90	2110.63	6.09	0.0838	0.71
SendPay	58	779	333.88	1416.28	4.30	0.0505	0.68
SerBG	14	365	365.43	1772.70	4.86	0.0623	0.75
SheriDroid	2	30	117.79	533.07	4.65	0.0925	0.25
SmForw	2	3	32.00	74.33	2.37	0.1621	0.07
SmsSpy	1	3	53.00	203.00	3.99	0.1872	0.11
SmsWatcher	3	15	109.60	332.00	3.03	0.0563	0.22
Sonus	1	1	13.00	24.00	1.85	0.3077	0.03
Spitmo	11	44	22.00	52.25	2.52	0.2821	0.05
SpyBubble	3	58	542.52	4001.97	6.57	0.0666	1.14
SpyHasb	13	65	67.22	422.48	6.25	0.2459	0.14
SpyMob	3	51	256.72	903.10	3.52	0.0278	0.52

Appendix B

Additional Experimental Details

B.1 Hashes of Synthetic Dataset Base APKs

This section lists the SHA-256 hashes of the base APKs used in the synthetic dataset for reproducibility and verification purposes.

```
040c0f05c85ccced9e999b6909c01d4bc109f17ba7644e276eb1341092007a22
1c6ca7ade89cc6715be60de6803c83ba19ff20403d366e23425c3995a4511f1e
2ee114a2d15db57b274899d510ea70d11ee54e4b1a39154aea1af2fc7c0c3a21
35a4befde72473731a2178eedc41fd443c1da7b2ebe6b8f1858b40f110aaba29
8d8c6712ec0c708792dba075f6a7995ff5124f13d31e8a0fbcbeeb0715b1f48e
9edc0d9959b59c4e6624cd0420d19277aafd09ce273e0816caf3a4fcf9d7342e
a45120136953fe4cd72bf952dd0d8f83218443e295dacc764dac5b2b66fe126c
d0b963b9983e7813a4a02f6bf37288cd8dfabfb240b783b8be9caec10b3a2f68
d918b2957c847ab4cb53dcff9db8f7cbd6e0dfa6a454f36865c526942e018715
ed0f0d2508eb7a2a3824fef24c1c432a751eb904c9c7afae202e6b60fd96192f
```

Bibliography

- [1] Marco Alecci et al. "AndroZoo: A Retrospective with a Glimpse into the Future". In: *Proceedings of the 21st International Conference on Mining Software Repositories*. 2024, pp. 389–393.
- [2] Kevin Allix et al. "AndroZoo: Collecting Millions of Android Apps for the Research Community". In: *Proceedings of the 13th International Conference on Mining Software Repositories*. MSR '16. Austin, Texas: ACM, 2016, pp. 468–471. ISBN: 978-1-4503-4186-8. DOI: [10.1145/2901739.2903508](https://doi.org/10.1145/2901739.2903508). URL: <http://doi.acm.org/10.1145/2901739.2903508>.
- [3] *Androguard*. <https://github.com/androguard/androguard>. Accessed: 2025-05-03.
- [4] *APKTool*. <https://ibotpeaches.github.io/Apktool/>. Accessed: 2025-05-03.
- [5] Daniel Arp et al. "Drebin: Effective and explainable detection of android malware in your pocket." In: 2014.
- [6] Kathy Wain Yee Au et al. "PScout: analyzing the Android permission specification". In: *Proceedings of the 2012 ACM Conference on Computer and Communications Security*. CCS '12. Raleigh, North Carolina, USA: Association for Computing Machinery, 2012, pp. 217–228. ISBN: 9781450316514. DOI: [10.1145/2382196.2382222](https://doi.org/10.1145/2382196.2382222). URL: <https://doi.org/10.1145/2382196.2382222>.
- [7] Sebastian Bach et al. "On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation". In: *PLOS ONE* 10.7 (2015), e0130140. DOI: [10.1371/journal.pone.0130140](https://doi.org/10.1371/journal.pone.0130140). URL: <https://doi.org/10.1371/journal.pone.0130140>.
- [8] Michael Backes et al. "On demystifying the android application framework: re-visiting android permission specification analysis". In: *Proceedings of the 25th USENIX Conference on Security Symposium*. SEC'16. Austin, TX, USA: USENIX Association, 2016, pp. 1101–1118. ISBN: 9781931971324.
- [9] Tristan Bilot et al. "A Survey on Malware Detection with Graph Representation Learning". In: *ACM Comput. Surv.* 56.11 (June 2024). ISSN: 0360-0300. DOI: [10.1145/3664649](https://doi.org/10.1145/3664649). URL: <https://doi.org/10.1145/3664649>.
- [10] Hanjun Dai et al. *Adversarial Attack on Graph Structured Data*. 2018. arXiv: [1806.02371](https://arxiv.org/abs/1806.02371) [cs.LG]. URL: <https://arxiv.org/abs/1806.02371>.
- [11] Fatemeh Deldar, Mahdi Abadi, and Mohammad Ebrahimifard. "Android Malware Detection Using One-Class Graph Neural Networks". In: *The ISC International Journal of Information Security* 14.3 (2022), pp. 51–60. ISSN: 2008-2045. DOI: [10.22042/isecure.2022.14.3.6](https://www.isecure-journal.com/article_159681_56ce65dea909f10fe5e79c896e9d4133.pdf). eprint: https://www.isecure-journal.com/article_159681_56ce65dea909f10fe5e79c896e9d4133.pdf. URL: https://www.isecure-journal.com/article_159681.html.
- [12] Pengbin Feng et al. "Android Malware Detection via Graph Representation Learning". In: *Mob. Inf. Syst.* 2021 (Jan. 2021). ISSN: 1574-017X. DOI: [10.1155/2021/5538841](https://doi.org/10.1155/2021/5538841). URL: <https://doi.org/10.1155/2021/5538841>.
- [13] Han Gao, Shaoyin Cheng, and Weiming Zhang. "GDroid: Android malware detection and classification with graph convolutional network". In: *Comput. Secur.* 106.C (July

- 2021). ISSN: 0167-4048. DOI: [10.1016/j.cose.2021.102264](https://doi.org/10.1016/j.cose.2021.102264). URL: <https://doi.org/10.1016/j.cose.2021.102264>.
- [14] Jintao Gu et al. "GSEDroid: GNN-based Android malware detection framework using lightweight semantic embedding". In: *Comput. Secur.* 140.C (May 2024). ISSN: 0167-4048. DOI: [10.1016/j.cose.2024.103807](https://doi.org/10.1016/j.cose.2024.103807). URL: <https://doi.org/10.1016/j.cose.2024.103807>.
- [15] Y. He et al. "MsDroid: Identifying Malicious Snippets for Android Malware Detection". In: *IEEE Transactions on Dependable and Secure Computing* 20.3 (May 2023), pp. 2025–2039. DOI: [10.1109/TDSC.2022.3168285](https://doi.org/10.1109/TDSC.2022.3168285).
- [16] Ankita Kapratwar, Fabio Di Troia, and Mark Stamp. "Static and Dynamic Analysis of Android Malware". In: *Proceedings of the 2017 International Conference on Software Engineering Research and Practice (SERP)*. 2017, pp. 1–7.
- [17] Thomas N. Kipf and Max Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. 2017. arXiv: [1609.02907](https://arxiv.org/abs/1609.02907) [cs.LG]. URL: <https://arxiv.org/abs/1609.02907>.
- [18] Junjie Li et al. "DeepMalware: GNN-Based Android Malware Detection Using Permission-Induced Call Graphs". In: *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2021, pp. 1–6. DOI: [10.1109/GLOBECOM46510.2021.9685733](https://doi.org/10.1109/GLOBECOM46510.2021.9685733).
- [19] Wai Weng Lo et al. "Graph Neural Network-based Android Malware Classification with Jumping Knowledge". In: *2022 IEEE Conference on Dependable and Secure Computing (DSC)*. 2022, pp. 1–9. DOI: [10.1109/DSC54232.2022.9888878](https://doi.org/10.1109/DSC54232.2022.9888878).
- [20] Helena Löfström, Karl Hammar, and Ulf Johansson. "A Meta Survey of Quality Evaluation Criteria in Explanation Methods". In: *Intelligent Information Systems*. Springer International Publishing, 2022, pp. 55–63. ISBN: 9783031074813. DOI: [10.1007/978-3-031-07481-3_7](https://doi.org/10.1007/978-3-031-07481-3_7). URL: http://dx.doi.org/10.1007/978-3-031-07481-3_7.
- [21] Harikha Manthena et al. *Explainable Artificial Intelligence (XAI) for Malware Analysis: A Survey of Techniques, Applications, and Open Challenges*. 2025. DOI: <https://doi.org/10.1109/ACCESS.2025.3555926>. arXiv: [2409.13723](https://arxiv.org/abs/2409.13723) [cs.CR]. URL: <https://arxiv.org/abs/2409.13723>.
- [22] Enrico Mariconti et al. "MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models". In: *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*. 2017.
- [23] Jiaming Mu et al. *A Hard Label Black-box Adversarial Attack Against Graph Neural Networks*. 2021. arXiv: [2108.09513](https://arxiv.org/abs/2108.09513) [cs.LG]. URL: <https://arxiv.org/abs/2108.09513>.
- [24] M. Nauta et al. "From Anecdotal Evidence to Quantitative Evaluation Methods: A Systematic Review on Evaluating Explainable AI". In: *ACM Computing Surveys* 55.13s (July 2023), pp. 1–42. DOI: [10.1145/3583558](https://doi.org/10.1145/3583558).
- [25] Meike Nauta et al. "From Anecdotal Evidence to Quantitative Evaluation Methods: A Systematic Review on Evaluating Explainable AI". In: *ACM Comput. Surv.* 55.13s (July 2023). ISSN: 0360-0300. DOI: [10.1145/3583558](https://doi.org/10.1145/3583558). URL: <https://doi.org/10.1145/3583558>.
- [26] P. E. Pope et al. "Explainability Methods for Graph Convolutional Neural Networks". In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Long Beach, CA, USA, 2019, pp. 10764–10773. DOI: [10.1109/CVPR.2019.01103](https://doi.org/10.1109/CVPR.2019.01103).
- [27] L. Rui and Olga Gadyatskaya. "Position: The Explainability Paradox - Challenges for XAI in Malware Detection and Analysis". In: *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 2024, pp. 554–561. DOI: [10.1109/EuroSPW61312.2024.00067](https://doi.org/10.1109/EuroSPW61312.2024.00067).

- [28] Jordan Samhi et al. "Call Graph Soundness in Android Static Analysis". In: *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*. 2024, pp. 945–957. DOI: [10.1145/3650212.3680333](https://doi.org/10.1145/3650212.3680333).
- [29] T. Schnake et al. "Higher-Order Explanations of Graph Neural Networks via Relevant Walks". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.11 (Nov. 2022), pp. 7581–7596. DOI: [10.1109/TPAMI.2021.3115452](https://doi.org/10.1109/TPAMI.2021.3115452).
- [30] StatCounter Global Stats. *Mobile Operating System Market Share Worldwide*. Accessed: 2025-05-01. 2025. URL: <https://gs.statcounter.com/os-market-share/mobile/worldwide>.
- [31] Shuai Wang et al. "GraphDroid: Graph-based Android Malware Detection Using Contextual Relationships between Program Entities". In: *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies (SACMAT)*. ACM, 2020, pp. 97–108. DOI: [10.1145/3381991.3395607](https://doi.org/10.1145/3381991.3395607).
- [32] A. Warnecke et al. "Evaluating Explanation Methods for Deep Learning in Security". In: *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. Genoa, Italy, 2020, pp. 158–174. DOI: [10.1109/EuroSP48549.2020.00018](https://doi.org/10.1109/EuroSP48549.2020.00018).
- [33] Fangfang Wei et al. "Deep Ground Truth Analysis of Current Android Malware". In: *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer. 2017, pp. 252–276.
- [34] Zonghan Wu et al. "A comprehensive survey on graph neural networks". In: *IEEE Transactions on Neural Networks and Learning Systems* 32.1 (2021), pp. 4–24. DOI: [10.1109/TNNLS.2020.2978386](https://doi.org/10.1109/TNNLS.2020.2978386).
- [35] Ning Xi et al. "GNNDroid: Graph-Learning Based Malware Detection for Android Apps With Native Code". In: *IEEE Transactions on Dependable and Secure Computing* 22.2 (2025), pp. 1460–1476. DOI: [10.1109/TDSC.2024.3444913](https://doi.org/10.1109/TDSC.2024.3444913).
- [36] Keyulu Xu et al. "How Powerful are Graph Neural Networks?" In: *Proceedings of the 7th International Conference on Learning Representations (ICLR 2019)*. 2019. URL: <https://arxiv.org/abs/1810.00826>.
- [37] Zi Ye et al. "A Comprehensive Survey of Graph Neural Networks for Knowledge Graphs". In: *IEEE Access* 10 (2022), pp. 75729–75741. DOI: [10.1109/ACCESS.2022.3191784](https://doi.org/10.1109/ACCESS.2022.3191784).